COLLEGE DE PARIS SUPÉRIEUR

NIVEAU : B1 - IT

Année académique 2025 - 2026

BASE DE DONNEES & SQL

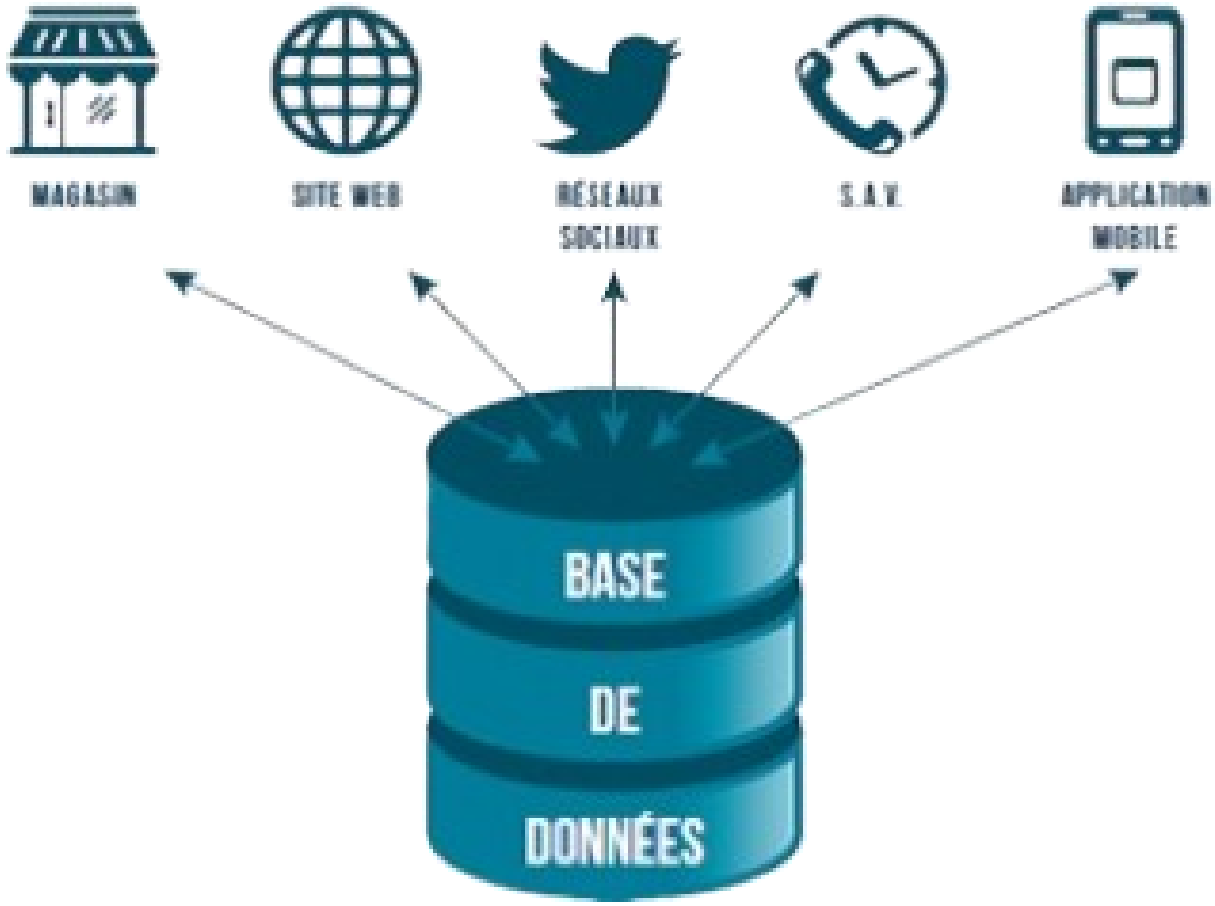
Simon Pierre EDOH

simon.edoh@gmail.com

90 16 55 18

GENERALITES SUR LES BASES DE DONNEES

CHAPITRE 1



I. QU'EST-CE QU'UNE BASE DE DONNÉES ?

En informatique et plus particulièrement en informatique de gestion, les bases de données ont de nos jours pris une place primordiale. Au cours des cinquante dernières années, des concepts, méthodes et algorithmes ont été développés pour gérer des données sur mémoires secondaires (disque dur, disquette, bandes magnétiques, etc.) ; ils constituent aujourd'hui l'essentiel de la discipline « Bases de Données » (BD). Cette discipline est utilisée dans de nombreuses applications. Il existe un grand nombre de Systèmes de Gestion de Bases de Données (SGBD) qui permettent de gérer efficacement de grandes bases de données.

De plus, une théorie fondamentale sur les techniques de modélisation des données et les algorithmes de traitement a vu le jour. Les bases de données constituent donc une discipline s'appuyant sur une théorie solide et offrant de nombreux débouchés pratiques.
Vous avez sans doute une idée intuitive des bases de données. Prenez garde cependant, car ce mot est souvent utilisé pour désigner n'importe quel ensemble de données ; il s'agit là d'un abus de langage qu'il faut éviter.

A. DEFINITION

Une base de données BDD ou encore database en anglais par définition est un ensemble d'informations qui est organisé pour être accessible, géré et mis à jour facilement par ses utilisateurs. Les données sont organisées en lignes, colonnes et tableaux et indexées pour faciliter les recherches. Les entreprises utilisent les bases de données pour stocker, gérer et récupérer des informations. Ces informations concernent une activité ou des activités bien données et peuvent être utilisées par des programmes ou des utilisateurs communs.
Une base de données peut être locale (utilisable par une machine, un utilisateur) ou bien répartie (informations stockées sur des machines distantes et accessibles par réseau). L'avantage majeur d'une base de données hormis de regrouper un grand nombre d'informations demeure la possibilité d'y accéder par plusieurs utilisateurs simultanément.

Pour mériter le terme de base de données, un ensemble de données non indépendantes doit être interrogeable par le contenu, c'est-à-dire que l'on doit pouvoir retrouver tous les objets qui satisfont à un certain critère, par exemple tous les produits qui coûtent moins de 100 francs. Les données doivent être interrogeables selon n'importe quel critère. Il doit être possible aussi de retrouver leur structure, par exemple le fait qu'un produit possède un nom, un prix et une quantité.

B. FONCTIONS D'UNE BASE DE DONNEES

Une base de données est structurée afin de pouvoir mieux répondre à des fonctions fondamentales en informatique, telles que :

- Stocker l'information de façon fiable (c'est à dire être capable de restituer l'information entrée dans le système)
- Traiter de grands volumes de données (massification)
- Traiter rapidement les données (optimisation)
- Sécuriser les accès aux données (gérer les autorisations selon les utilisateurs)
- Contrôler la qualité des données (par exemple la cohérence par rapport à un modèle préétabli)
- Partager les données (entre plusieurs applications dédiées à plusieurs métiers)
- Rendre accessible les données en réseau (gérer la concurrence des accès parallèles

II. LES DIFFERENTS TYPES DE BASES DE DONNEES

A. LA BASE DE DONNEES RESEAU

Les bases de données réseau permettent des relations multiples entre les données grâce à des liens ou pointeurs. Elles sont rapides, flexibles et utilisées notamment dans le e-commerce.

B. LA BASE DE DONNEES HIERARCHIQUE

Les bases de données hiérarchiques organisent les données sous forme d'arborescence. Chaque enregistrement dépend d'un niveau supérieur, selon une structure logique hiérarchisée.

C. LA BASE DE DONNEES ORIENTEE OBJETS

Les données sont stockées sous forme d'objets avec leurs attributs, comme en programmation orientée objet. Bien qu'adaptées à ce modèle, ces bases sont peu utilisées en raison de leur complexité.

D. LA BASE DE DONNEES SQL OU RELATIONNELLE

Les bases relationnelles sont les plus répandues. Elles organisent les données en tables (lignes et colonnes) et utilisent le langage SQL. Elles sont simples à maintenir et évolutives.

E. LA BASE DE DONNEES NOSQL

Les bases NoSQL sont conçues pour le Big Data. Elles gèrent efficacement de grandes quantités de données non structurées et distribuées.

F. LA BASE DE DONNEES CLOUD

Les bases de données cloud fonctionnent dans des environnements virtualisés. Elles offrent une grande disponibilité, une mise à l'échelle flexible et un paiement selon l'usage.

G. LA BASE DE DONNEES ORIENTEE TEXTE

Aussi appelées fichiers plats, elles stockent les données dans des fichiers texte où chaque ligne correspond à un enregistrement. Elles sont souvent utilisées sur les ordinateurs personnels.

H. LA BASE DE DONNEES DISTRIBUEE

Les bases distribuées répartissent les données sur plusieurs sites physiques. Elles peuvent être homogènes (mêmes systèmes) ou hétérogènes (systèmes différents).

I. LA BASE DE DONNEES ORIENTEE GRAPHES

Les bases orientées graphes stockent les données sous forme de nœuds et de relations. Elles sont très efficaces pour analyser des connexions complexes, comme les réseaux sociaux ou la détection de fraudes.

LES SYSTEME DE GESTION DE BASE DE DONNÉES

CHAPITRE 2

I. DEFINITION

Un SGBD peut être perçu comme un ensemble de logiciels systèmes permettant aux utilisateurs d'insérer, de modifier et de rechercher efficacement des données spécifiques dans une grande masse d'informations (pouvant atteindre quelques milliards d'octets) partagée par de multiples utilisateurs. Les informations sont stockées sur mémoires secondaires, en général des disques magnétiques. Les recherches peuvent être exécutée à partir de la valeur d'une donnée désignée par un nom dans un ensemble d'objets (par exemple, les produits de prix inférieur à 100 francs), mais aussi à partir de relations entre objets (par exemple, les produits commandés par un client habitant Atakpamé). Les données sont partagées, aussi bien en interrogation qu'en mise à jour. Le SGBD rend transparent le partage, à savoir donne l'illusion à chaque utilisateur qu'il est seul à travailler avec les données.

En résumé, un SGBD peut donc apparaître comme un outil informatique permettant la sauvegarde, l'interrogation, la recherche et la mise en forme de données stockées sur mémoires secondaires. Ce sont là les fonctions premières, complétées par des fonctions souvent plus complexes, destinées par exemple à assurer le partage des données mais aussi à protéger les données contre tout incident et à obtenir des performances acceptables. Les SGBD se distinguent des systèmes de fichiers par le fait qu'ils permettent la description des données (définition des types par des noms, des formats, des caractéristiques et parfois des opérations) de manière séparée de leur utilisation (mise à jour et recherche). Ils permettent aussi de retrouver les caractéristiques d'un type de données à partir de son nom (par exemple, comment est décrit un produit). Le système de fichiers est un composant de plus bas niveau ne prenant pas en compte la structure des données. La tendance est aujourd'hui à intégrer le système de fichiers dans le SGBD, construit au-dessus.

II. HISTORIQUE DU SYSTEME DE GESTION DE BASE DE DONNEES (SGBD)

Les SGBD ont plus de soixante ans d'histoire. Les années 60 ont connu un premier développement des bases de données sous forme de fichiers reliés par des pointeurs.

Les fichiers sont composés d'articles stockés les uns à la suite des autres et accessibles par des valeurs de données appelées clés. Les systèmes IDS.I et IMS.I développés respectivement à Honeywell et à IBM vers 1965 pour les programmes de conquête spatiale, notamment pour le programme APOLLO qui a permis d'envoyer un homme sur la lune, sont les précurseurs des SGBD modernes. Ils permettent de constituer des chaînes d'articles entre fichiers et de parcourir ces chaînes.

Les premiers SGBD sont réellement apparus à la fin des années 60. La première génération de SGBD est marquée par la séparation de la description des données et de la manipulation par les programmes d'application. Elle coïncide aussi avec l'avènement des langages d'accès navigationnels, c'est-à-dire permettant de se déplacer dans des structures de type graphe et d'obtenir, un par un, des articles de fichiers. Cette première génération, dont l'aboutissement est marqué par les recommandations du CODASYL, est basée sur les modèles réseau ou hiérarchique, c'est-à-dire des modèles de données organisés autour de types d'articles constituant les nœuds d'un graphe, reliés par des types de pointeurs composant les arcs du graphe. Cette génération a été dominée par les SGBD TOTAL, IDMS, IDS 2 et IMS 2. Elle traite encore aujourd'hui une partie importante du volume de données gérées par des SGBD.

La deuxième génération de SGBD a grandi dans les laboratoires depuis 1970, à partir du modèle relationnel. Elle vise à enrichir mais aussi à simplifier le SGBD externe afin de faciliter l'accès aux données pour les utilisateurs. En effet, les données sont présentées aux utilisateurs sous forme de relations entre domaines de valeurs, simplement représentées par des tables. Les recherches et mises à jour sont effectuées à l'aide d'un langage non procédural standardisé appelé SQL (Structured Query Language). Celui-ci permet d'exprimer des requêtes traduisant directement des phrases simples du langage naturel et de spécifier les données que l'on souhaite obtenir sans dire comment les accéder. C'est le SGBD qui doit déterminer le meilleur plan d'accès possible pour évaluer une requête. Cette deuxième génération reprend, après les avoir faits évoluer et rendus plus souples, certains modèles d'accès de la première génération au niveau du SGBD interne, afin de mieux optimiser les accès. Les systèmes de deuxième génération sont commercialisés depuis 1980. Ils représentent aujourd'hui l'essentiel du marché des bases de données.

Les principaux systèmes sont : **ORACLE, INGRES, SYBASE, INFORMIX, DB2 et SQL SERVER, MYSQL, POSTGRESQL, MARIADB, ACCESS**.

Ils supportent en général une architecture répartie, au moins avec des stations clients transmettant leurs requêtes à de puissants serveurs gérant les bases.

La troisième génération a été développée dans les laboratoires depuis le début des années 80. Elle commence à apparaître fortement dans l'industrie avec les extensions objet des systèmes relationnels. Elle supporte des modèles de données extensibles intégrant le relationnel et l'objet, ainsi que des architectures mieux réparties, permettant une meilleure collaboration entre des utilisateurs concurrents.

III. FONCTIONS DU SGBD

Le Système de Gestion de Base de Données (**SGBD**) permet entre autres de :

- **Décrire les données** : indépendamment des applications et de manière intrinsèque ;
- **Manipuler les données** : interroger et mettre à jour les données ;
- **Contrôler les données** : comporte deux notions à savoir **Intégrité** (vérification de contraintes d'intégrité) et **Confidentialité** (contrôle des droits d'accès et autorisation) ;
- **Partager** : la BD est partagée entre plusieurs utilisateurs en même temps tout en contrôlant des accès concurrents ;
- **Sécuriser** : permettre une reprise après panne et la journalisation etc ;
- Garantir la Performance dans l'accès des données : gestion des index par exemple ;
- **Garantir une Indépendance physique :** pouvoir modifier les structures de stockage ou les index sans que cela ait de répercussion au niveau des applications ;
- **Garantir une Indépendance logique :** permettre aux différentes applications d'avoir des vues différentes des mêmes données et au DBA de modifier le schéma logique sans que cela ait de répercussion au niveau des applications.

LES REQUETES

CHAPITRE 3

I. DEFINITION

Nous avons vu que la plupart des SGBD offrent la possibilité d'effectuer des recherches directement dans les tables. Les possibilités de formuler des critères de recherche sont cependant souvent assez limitées. Heureusement, la plupart des SGBD nous offrent également la possibilité de poser pratiquement n'importe quelle "question" à nos tables, sous forme de requêtes.

Les requêtes servent donc à répondre aux questions basées sur le contenu d'une ou de plusieurs tables. Nous allons plus tard étudier des requêtes, qui se basent sur plusieurs tables, mais pour l'instant nous allons nous limiter aux questions simples basées sur une seule table.

Exemple :
Prenons l'exemple d'une table ETUDIANT :

MatEtud	NomEtud	PrenEtud	SexEtud	DatNEtud	TelEtud	NatEtud
20230009	BABA	Jean	M	28/03/2000	90000000	Nigérian
20230025	CORLAY	Amina	F	10/10/1999	98888888	Anglaise
20230040	DOUMEVI	Cindy	F	30/01/2004	90777777	Togolaise
20230100	TOMFAI	Abalo	M	27/07/2002		Togolais
20230576	KEITA	Jules	M	10/01/2002	99666666	Malien
20230580	MISSAMBO	Pauline	F	13/07/2003		Gabonaise
...	...	...	...	…		…

Une **requête simple** serait par exemple :

Quelles sont les MatEtud, PrenEtud et NatEtud des étudiants de sexe masculin ?

Le résultat serait un sous-ensemble de la table ETUDIANT avec seulement les enregistrements qui vérifient le critère de sélection (→ Sexe = M). Pour chacun de ces enregistrements, le SGBD affiche en plus seulement les champs explicitement demandés (→ **MatEtud, PrenEtud et NatEtud**).

Une requête simple produit donc comme résultat un sous-ensemble des enregistrements d'une table. En plus, une requête nous permet d'afficher seulement certains champs pour les enregistrements appartenant à ce sous-ensemble.

On appelle ces requêtes "Requêtes de Sélection", puisqu'il s'agit d'une sélection de certains enregistrements.

Il n'existe cependant pas seulement des requêtes de sélection, mais également :

Des **requêtes d'insertion** → insérer des enregistrements dans la table.

Exemple : Insérer un nouvel étudiant dans la table : 20230630, NAPO, Latifa, F, 21/06/2021, 98766666, Togolaise.
Des **requêtes de modification** → modifier des enregistrements dans la table.

Exemple : Modifier la nationalité de l'étudiant BABA de façon qu'il Tchadien.

Des **requêtes de suppression** → supprimer des enregistrements de la table.

Exemple : Supprimer tous les étudiant n'ayant pas fournis leur numéro de téléphone.

Les requêtes possèdent l'avantage de pouvoir manipuler facilement un grand nombre d'enregistrements sans que l'utilisateur ne doive s'occuper de sélectionner enregistrement par enregistrement. Il lui suffit de spécifier des critères de sélection pour la requête, ainsi que l'opération à effectuer (→ simple sélection et affichage, insertion, modification ou suppression).

Bien que les requêtes de sélection soient implémentées d'une manière plus ou moins cohérente à travers les SGBD actuels, il existe des différences subtiles en ce qui concerne les requêtes d'insertion, de modification ainsi que de suppression. En plus, l'insertion et la suppression se font souvent de manière plus facile directement dans la table.

II. TRAITEMENT DES REQUETES PAR LES SGBD

La chronologie des opérations dans l'interrogation d'un SGDB est la suivante :

Un programme ou une application A émet une demande de lecture de données au SGBD sur une des bases :

- Le SGBD traite la demande en consultant le sous-schéma externe relatif au programme d'application A, obtenant ainsi la description des données.
- Le SGBD consulte le schéma conceptuel et détermine le type logique de données à extraire.
- Le système examine la description physique de la base en rapport avec la requête logique et détermine-le (ou les) enregistrement(s) physique(s) à lire.
- Le système lance une commande au système d'exploitation pour rechercher physiquement l'enregistrement désiré.
- Le système d'exploitation, par le biais de ses méthodes d'accès, accède à l'enregistrement physique.
- Les données demandées sont transférées dans les buffers, ou mémoires tampons.
- Le SGBD, à partir d'une comparaison entre le schéma logique global (conceptuel) et le sous-schéma externe de l'application A, extrait des données stockées dans le buffer, l'enregistrement logique réclamé par le programme d'application. Il effectue également les transformations éventuelles de format.
- Le SGBD transfère les données des buffers dans la zone de liaison du programme d'application A.
- Le SGBD fournit également des informations "d'état" au programme d'application, lui signalant en particulier les erreurs éventuellement constatées au cours du processus d'extraction.
- Le programme d'application, qui dispose des données et d'informations de "service" en assure la bonne exploitation !

III. LE LANGAGE SQL

A. INTRODUCTION

Les serveurs de données relationnels présentent aujourd'hui une interface externe sous forme d'un langage de recherche et mise à jour, permettant de spécifier les ensembles de données à sélectionner ou à mettre à jour à partir de propriétés des valeurs, sans dire comment retrouver les données. Ainsi, les opérations directement utilisables par les usagers sont en général celles des langages dits assertionnels. Plusieurs langages assertionnels permettant de manipuler des bases de données relationnelles ont été proposés, en particulier QUEL, QBE et SQL.

Aujourd'hui, le langage SQL est normalisé [ISO89, ISO92] et constitue le standard d'accès aux bases de données relationnelles. Les autres interfaces par menus, fenêtres, grilles, etc., ou de programmation type langage de 3e ou 4e génération, sont le plus souvent offertes au-dessus du langage SQL. Celui-ci constitue donc le point d'entrée obligatoire des SGBD relationnels. QUEL était le langage proposé par l'université de Berkeley pour son système INGRES : il est aujourd'hui peu utilisé dans l'industrie. QBE est un langage par grille dérivé de la logique qui est souvent offert au-dessus de SQL.

De manière générale, SQL comme les autres langages qui ont été proposés (ex : QUEL) utilisent tous des critères de recherche (encore appelés qualifications) construits à partir de la logique des prédicats du premier ordre.

Le langage SQL comporte plusieurs sortes de commandes de base parmi lesquelles :

- **LDD (Langage de Définition de Données)** : Ensemble de commandes qui définit une base de données et les objets qui la composent. Il s'agit de :
  - Création : **CREATE**
  - Modification : **ALTER**
  - Suppression : **DROP**

- **LMD (Langage de Manipulation des Données),** il est composé du **LMJ** (***Langage de Mise à Jour***) et du **LID** (***Langage d'Interrogation des Données***) : Pour l'insertion, la consultation et la mise à jour et la suppression des données dans les objets crées avec LDD. Il s'agit de :
  - Insertion : **INSERT**
  - Consultation de données : **SELECT**
  - Mise à jour : **UPDATE**
  - Suppression : **DELETE**

- **LCD (Langage de Contrôle des Données)** : Ensemble de commandes de contrôle d'accès aux données. Le contrôle d'accès inclut :
  - L'autorisation à réaliser une opération : **GRANT**
  - L'interdiction de réaliser une opération : **DENY**
  - Annulation d'une commande de contrôle précédente : **REVOKE**
  - L'autorisation à modifier des enregistrements : **UPDATE**
  - L'interdiction de modifier des enregistrements : **READ**
  - L'autorisation à supprimer des enregistrements : **DELETE**

Ce cours est animé en utilisant MySQL. Le cours sera fait en mode hybride en combinant la théorie à la pratique. Les notions vues en théorie seront testées en pratiques sur la base de données MySQL qui sera installée sur la machine de chaque étudiant.

La conception et l'utilisation des bases de données étant un sujet très vaste, ce cours abordera entre autres les notions suivantes :

- Créer, utiliser et supprimer une base de données
- Créer et modifier des tables
- Insérer, modifier, supprimer et sélectionner des données dans la base
- Réaliser une requête portant sur plusieurs tables avec les jointures
- Manipuler les clés pour garantir la cohérence des données
- Utiliser des fonctions d'agrégation

B. NOTIONS DE MYSQL

MySQL est un des SGBDR les plus utilisés. Sa popularité est due en grande partie au fait qu'il s'agit d'un logiciel open source. Une version gratuite de MySQL est par conséquent disponible. À noter qu'une version commerciale payante existe également.

Le développement de MySQL commence en 1994 par David Axmark et Michael Widenius.
En 1995, la société MySQL AB est fondée par ces deux développeurs, et Allan Larsson. C'est la même année que sort la première version officielle de MySQL.
En 2008, MySQL AB est rachetée par la société Sun Microsystems, qui est elle-même rachetée par Oracle Corporation en 2010.

1. Installation de mysql

En vue de préparer la pratique, nous allons procéder à l'installation du SGBDR MySQL sur vos ordinateurs.

Les captures d'écran suivantes montrent les étapes essentielles de l'installation de MySQL sous Windows. Ces captures peuvent être différentes suivant la version de MySQL choisie.

La version de MySQL utilisée pour les captures suivants est « mysql-installer-community-8.0.30.0 ».

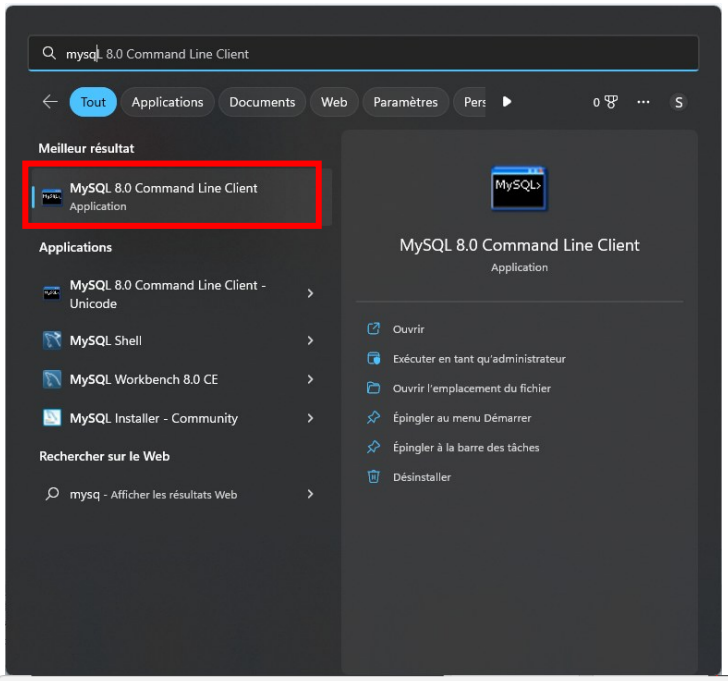
2. Utilisation de mysql

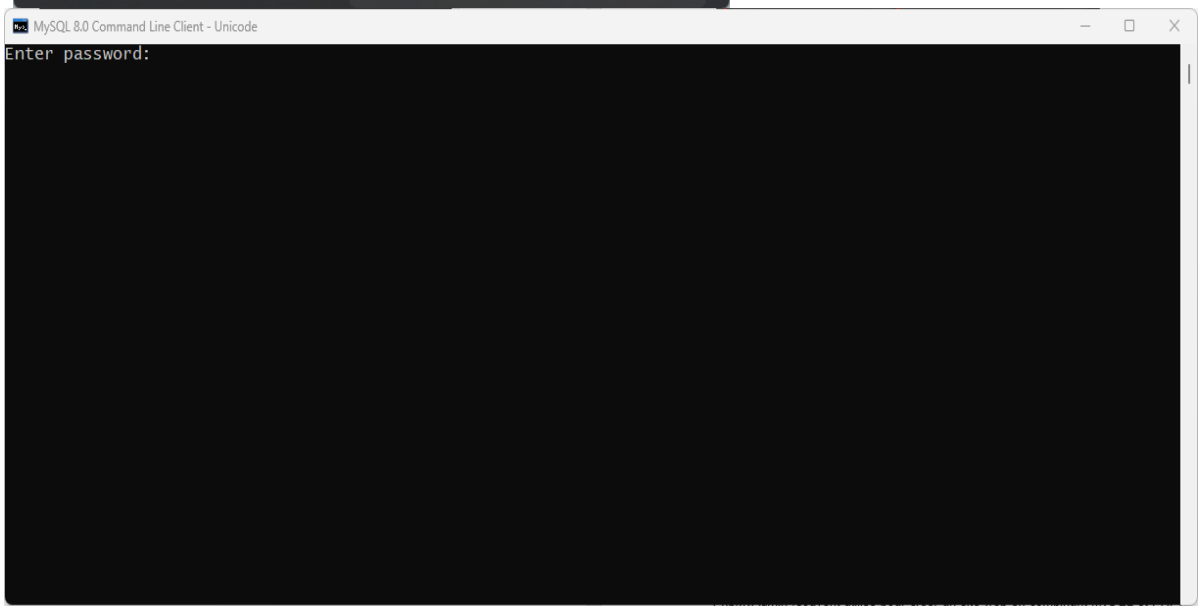
Il existe plusieurs manières de lancer MySQL pour l'utiliser.

a. En ligne de commande

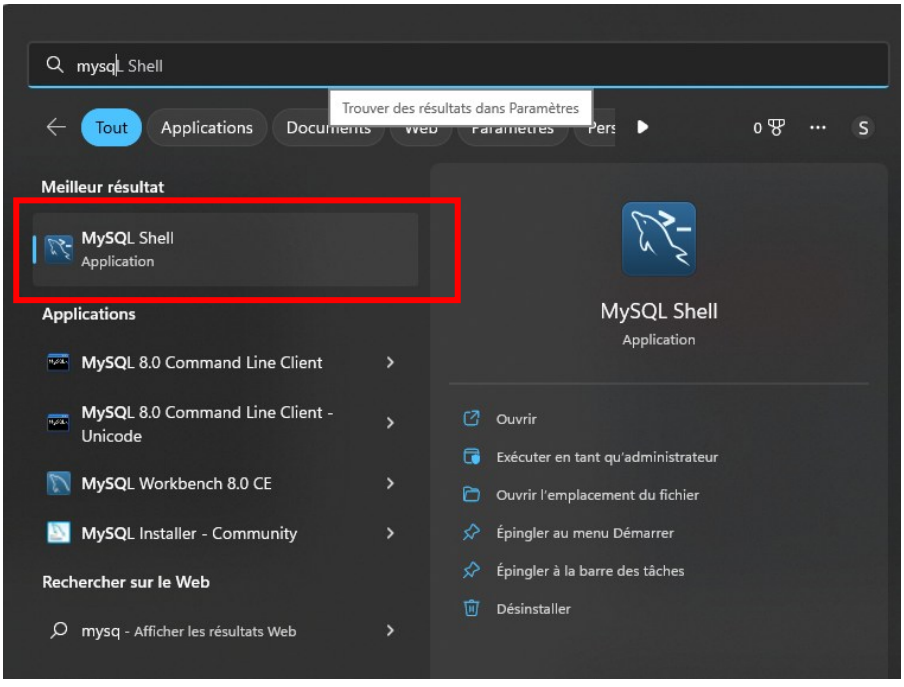
Il s'agit d'une fenêtre de dos dans laquelle toutes les instructions sont tapées à la main. Pas de bouton, pas de zone de saisie. Juste en utilisant votre clavier : c'est celle-ci qui sera utilisée au cours de nos travaux pratiques.

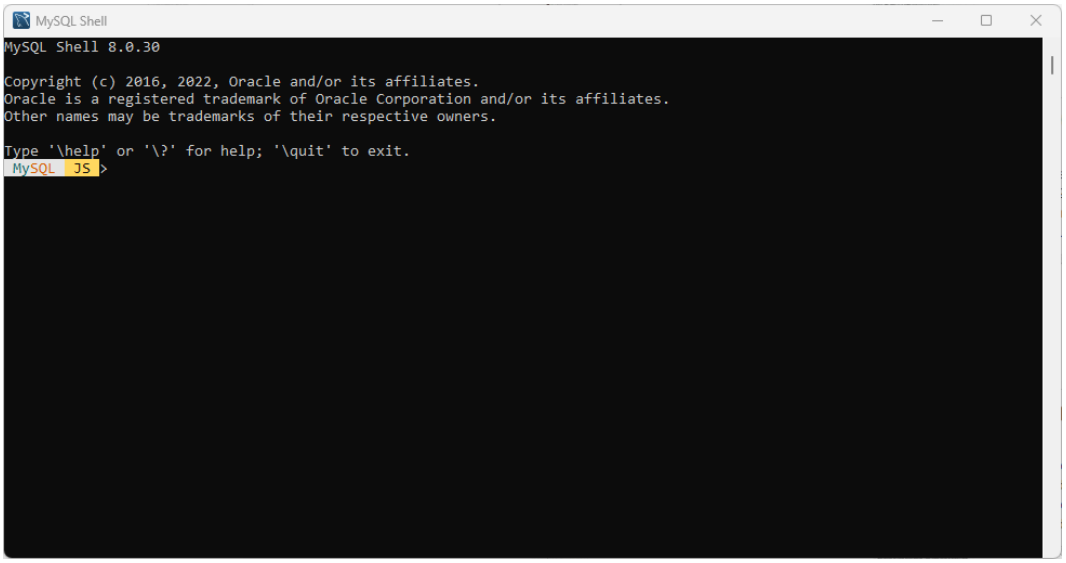
Lancement de la console de ligne de commandes sous Dos





MySQL dispose elle-même aussi de sa propre console d'utilisation qui est « MySQL Shell »

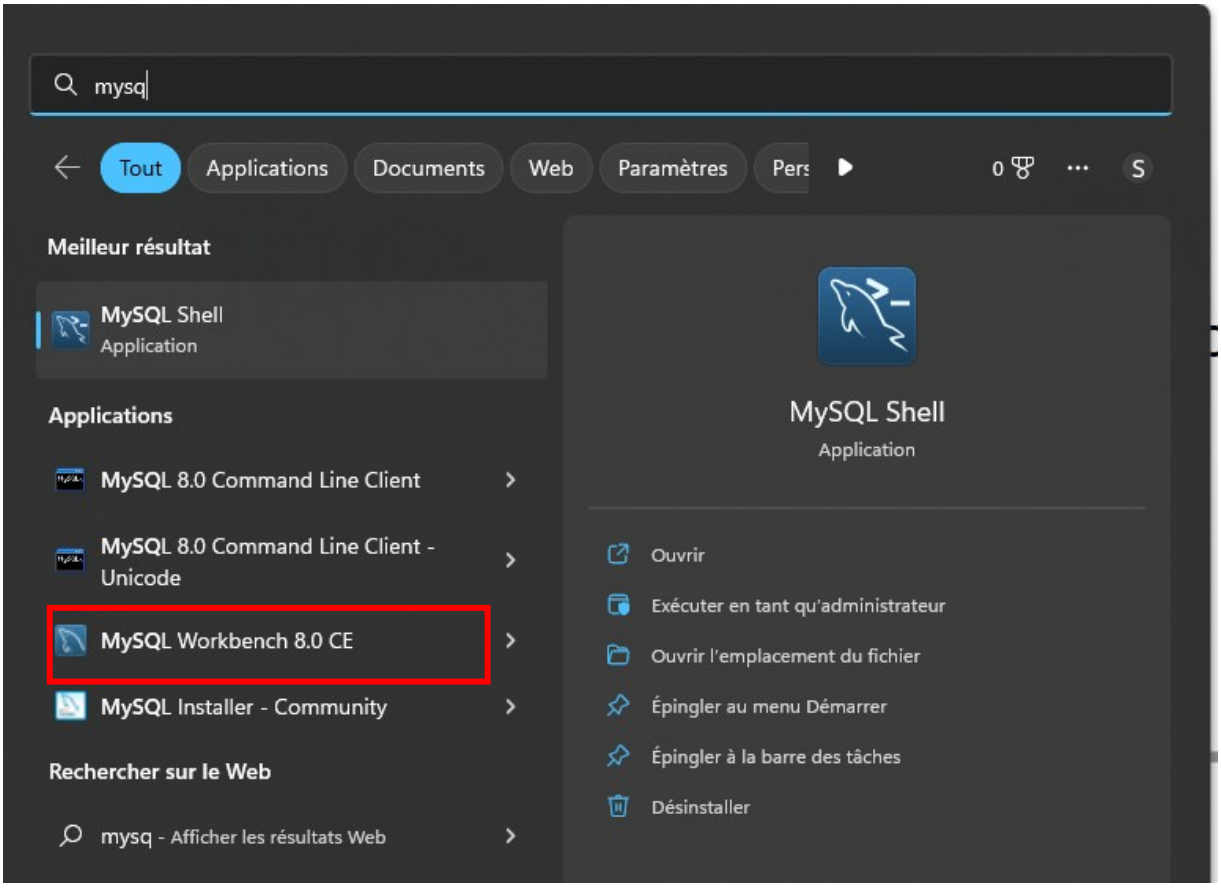


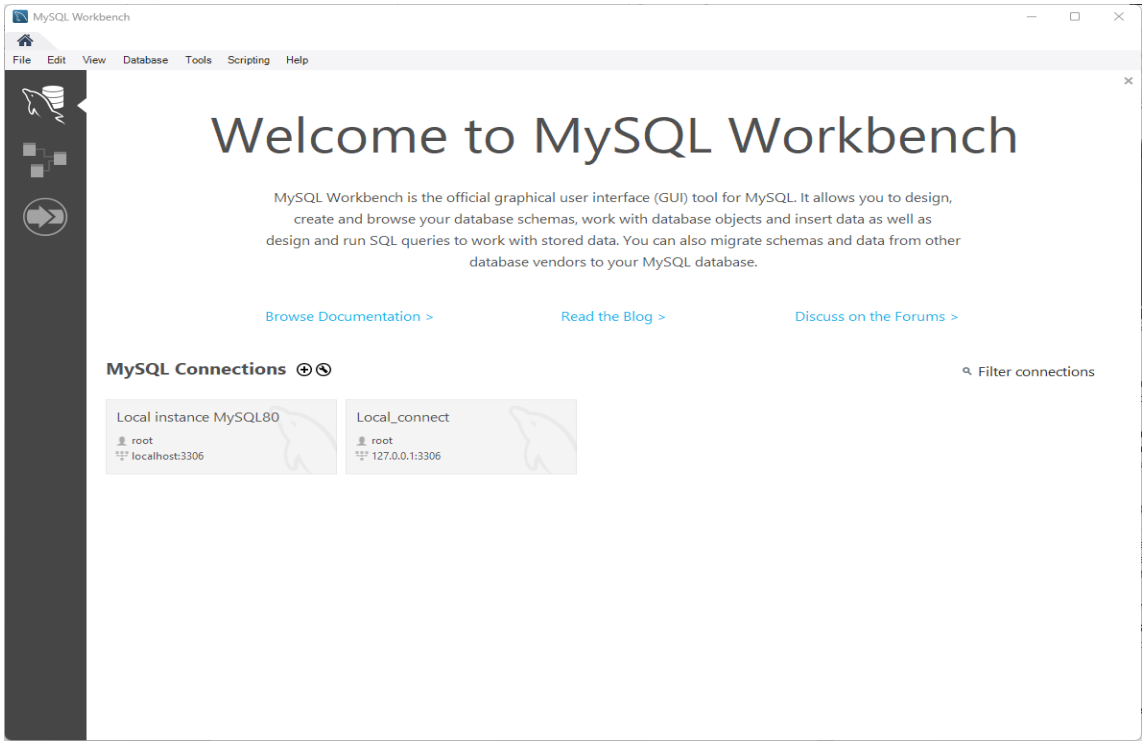


b. Via interface graphique :

Si l'on ne veut pas utiliser la ligne de commande (*il faut bien avouer que ce n'est pas très sympathique cette fenêtre monochrome*), on peut utiliser une interface graphique qui permet d'exécuter pas mal de choses simples de manière intuitive sur une base de données.

Comme interfaces graphiques pour MySQL, on peut citer MySQL Workbench, PhpMyAdmin (souvent utilisé pour créer un site web en combinant MySQL et PHP) ou MySQL Front etc.





Pourquoi utiliser la ligne de commande ?

Deux raisons :

- Permet à l'étudiant de maîtriser vraiment les commandes et d'avoir une idée de ce qui se passe sous les interfaces graphiques
- Parce qu'il est fort probable que vous désiriez utiliser MySQL en combinaison avec un autre langage de programmation. Or, dans du code PHP (ou Java, ou Python, etc.), vous allez devoir écrire en dur vos requêtes. Il faut donc que vous sachiez comment faire.

c. Connexion et déconnexion à mysql

MySQL est basé sur un modèle client-serveur, comme la plupart des SGBD. Cela implique donc que la base de données se trouve sur un serveur auquel vous n'avez pas accès directement, il faut obligatoirement passer par un client qui fera la liaison entre vous et le serveur.

Après l'installation de MySQL, plusieurs choses sont installées sur votre ordinateur à savoir :

- un serveur de base de données MySQL ;
- plusieurs logiciels clients qui permettent d'interagir avec le serveur.

Pour la première connexion à MySQL, il faudra se connecter avec l'utilisateur root, pour lequel un mot de passe a été normalement défini pendant l'installation. Par la suite, un nouvel utilisateur pourra être créé.
Ainsi pour se connecter, on aura besoin de renseigner les trois paramètres suivants : le serveur, le client (utilisateur) et son mot de passe.

Pour chacun de ces trois paramètres utiles pour la connexion, deux syntaxes sont possibles :

```
mysql -h localhost -u root -p motdepassetopsecret    # ou

mysql --host=localhost --user=root --password=motdepassetopsecret
```

```
# ou un mélange des paramètres courts et longs

mysql -h localhost --user=root -p motdepassetopsecret
```

Si vous utilisez les paramètres courts comme ceci, `mysql -h localhost -u root -p` avec : h pour host, u pour username, p pour password.

Vous aurez le prompt `Enter password:` qui vous demandera de taper le mot de passe.

Déconnexion : pour se déconnecter du client, il suffit d'utiliser la commande `quit` ou `exit`.

Maintenant que vous savez vous connecter, vous allez enfin pouvoir discuter avec le serveur MySQL (en langage SQL évidemment). Donc, reconnectez-vous si vous êtes déconnecté.

Vous pouvez constater que vous êtes connecté. Au changement de l'invite de commande, vous verrez maintenant `mysql>` .

Taper par exemple les commandes suivantes (sans oublier le « ; » à la fin) :

```
SELECT 'Nous sommes au COLLEGE DE PARIS SUPERIEUR' ;
SELECT 45+40 as Effectif ;
SELECT 'MES MEILLEURS VOEUX DE NOUVEL AN' as Vœux ;
```

SELECT est la commande qui permet la sélection de données, mais aussi l'affichage. Pour les commandes ci-dessus, vous aurez les résultats suivants :

d. Quelques syntaxes

Avant d'aller plus loin, voici quelques règles générales à retenir concernant le SQL qui, comme tout langage informatique, obéit à des règles syntaxiques très strictes.

Fin d'une instruction :

Dans MySQL, une instruction doit être explicitement terminée pour être exécutée. Deux options permettent d'indiquer à MySQL que l'instruction est terminée :

- Le point-virgule (`;`)
- La commande `\g`

✓ **Le rôle du point-virgule (`;`) :**

Le point-virgule est le séparateur de fin d'instruction par défaut dans MySQL. Tant que MySQL ne rencontre pas ce caractère, il considère que vous n'avez pas terminé d'écrire votre commande.

L'utilisation de ; permet :

- **D'écrire une instruction sur plusieurs lignes :**
  Vous pouvez répartir une seule commande sur plusieurs lignes sans problème. Par exemple :

```
SELECT nom, prenom
FROM utilisateurs
WHERE age > 25;
```

- **D'écrire plusieurs instructions sur une seule ligne :**
  Plusieurs commandes peuvent être séparées par un point-virgule sur une seule ligne. Par exemple :

  USE ma_base; SELECT COUNT(*) FROM utilisateurs;

✓ **La commande \g comme alternative :**

\g est une commande spécifique à l'interface en ligne de commande MySQL. Elle joue un rôle similaire au point-virgule en indiquant que l'instruction est terminée et doit être exécutée.

Cas d'utilisation de \g :

- **Elle peut remplacer le point-virgule à la fin d'une commande :**

  SELECT * FROM clients WHERE age > 30 \g

  Cette syntaxe est particulièrement utile dans le client MySQL pour signaler explicitement la fin d'une instruction.

- **Redirection des résultats dans un fichier :**
  Avec \g, il est possible d'enregistrer directement les résultats d'une commande dans un fichier, ce qui n'est pas faisable avec ;. Par exemple :

  SELECT * FROM clients \g resultats.txt

  Ici, les résultats de la requête seront enregistrés dans le fichier resultats.txt.

Commentaires : Les commentaires sont des parties de code qui ne sont pas interprétées. Ils servent principalement à vous repérer dans votre code. En SQL, les commentaires sont introduits par `--` (deux tirets).

Cependant, MySQL déroge un peu à la règle SQL et accepte deux syntaxes :

- `#` : tout ce qui suit ce caractère sera considéré comme commentaire.
- `--` : la syntaxe normale est acceptée uniquement si les deux tirets sont suivis d'une espace au moins.

Chaînes de caractères : Lorsque vous écrivez une chaîne de caractères dans une commande SQL, il faut absolument l'entourer de guillemets simples (*donc des apostrophes*).
MySQL permet également l'utilisation des guillemets doubles, mais ce n'est pas le cas de la plupart des SGBDR.

Par ailleurs, si vous désirez utiliser un caractère spécial dans une chaîne, il vous faudra l'échapper avec `\`.

Exemple : Utilisation de guillemet à l'intérieur de votre chaîne

```
SELECT 'Salut l'ami'; -- Pas bien !
SELECT 'Salut l\'ami'; -- Bien !
```

Quelques autres caractères spéciaux :

Caractère	Explication
\n	retour à la ligne
\t	tabulation
\\	antislash (il faut échapper le caractère d'échappement…)
%	pourcent (vous verrez pourquoi plus tard)
_	souligné (vous verrez pourquoi plus tard aussi)

Cette manière d'échapper les caractères spéciaux (avec \) est propre à MySQL.

Notez que, pour échapper un guillemet simple (et uniquement ce caractère), vous pouvez également l'écrire deux fois. Cette façon d'échapper les guillemets correspond d'ailleurs à la norme SQL.

Exemple :

```
SELECT 'Salut l'ami';  -- ne fonctionne pas !
SELECT 'Salut l\'ami'; -- fonctionne !
SELECT 'Salut l''ami'; -- fonctionne aussi et correspond à la norme !
```

3. Différents types de données (exemple de mysql)

En SQL (et dans la plupart des langages informatiques), les données sont séparées en plusieurs types (*par exemple : texte, nombre entier, date*…). Lorsque l'on définit une colonne dans une table de la base, il faut donc lui donner un type, et toutes les données stockées dans cette colonne devront correspondre au type de la colonne.

Il est important de bien comprendre les usages et particularités de chaque type de données, afin de choisir ***le meilleur type possible*** lorsque vous définissez les colonnes de vos tables. En effet, choisir un mauvais type de donnée pourrait entraîner :

- **un gaspillage de mémoire** (si vous stockez de toutes petites données dans une colonne faite pour stocker de grosses quantités de données) ;
- **des problèmes de performance** (il est plus rapide de faire une recherche sur un nombre que sur une chaîne de caractères) ;
- **un comportement contraire à celui attendu** (trier sur un nombre stocké comme tel, ou sur un nombre stocké comme une chaîne de caractères ne donnera pas le même résultat) ;
- **l'impossibilité d'utiliser des fonctionnalités propres à un type de données** (stocker une date comme une chaîne de caractères vous prive des nombreuses fonctions temporelles disponibles).

a. *Types numériques*

On peut subdiviser les types numériques en deux sous-catégories : les nombres entiers, et les nombres décimaux.

i. *Nombres entiers*

Type	Nombre d'octets	Minimum	Maximum
TINYINT	1	-128	127
SMALLINT	2	-32768	32767
MEDIUMINT	3	-8388608	8388607
INT	4	-2147483648	2147483647
BIGINT	8	-9223372036854775808	9223372036854775807

L'attribut UNSIGNED : Vous pouvez également préciser que vos colonnes sont **UNSIGNED** (on aura donc toujours une valeur positive). Dans ce cas, la longueur de l'intervalle reste la même, mais les valeurs possibles sont décalées, le minimum valant 0. Pour les TINYINT, on pourra par exemple aller de 0 à 255.

Limiter la taille d'affichage et l'attribut ZEROFILL : Il est possible de préciser le nombre de chiffres minimum à l'affichage d'une colonne de type INT(ou un de ses dérivés). Il suffit alors de préciser ce nombre entre parenthèses : **INT(x)**. Notez bien que cela ne change pas les capacités de stockage dans la colonne. Si vous déclarez un INT(2), vous pourrez toujours y stocker 45282.
Simplement, si vous stockez un nombre avec un nombre de chiffres inférieur au nombre défini, le caractère par défaut sera ajouté à gauche du chiffre, pour qu'il prenne la bonne taille. Sans précision, le caractère par défaut est l'espace.

Cette taille d'affichage est généralement utilisée en combinaison avec l'attribut **ZEROFILL**. Cet attribut ajoute des zéros à gauche du nombre lors de son affichage, il change donc le caractère par défaut par '0'. Donc si vous déclarez une colonne comme étant

```
INT(4) ZEROFILL
```

Vous aurez l'affichage suivant :

Nombre stocké	Nombre affiché
45	0045
4156	4156
785164	785164

ii. *Nombres décimaux*

Cinq mots-clés permettent de stocker des nombres décimaux dans une colonne : **DECIMAL**, **NUMERIC**, **FLOAT**, **REAL** et **DOUBLE**.

- **NUMERIC et DECIMAL**

NUMERIC et DECIMAL sont équivalents et acceptent deux paramètres : la précision et l'échelle.

- La précision définit le nombre de chiffres significatifs stockés, donc les 0 à gauche ne comptent pas. En effet, 0024 est équivalent à 24. Il n'y a donc que deux chiffres significatifs dans 0024.
- L'échelle définit le nombre de chiffres après la virgule.

Dans un champ DECIMAL (5,3), on peut donc stocker des nombres de 5 chiffres significatifs au maximum, dont 3 chiffres sont après la virgule. Par exemple : 12.354, -54.258, 89.2 ou 56. DECIMAL (4) équivaut à écrire DECIMAL (4, 0).

- **FLOAT, DOUBLE et REAL**

Le mot-clé **FLOAT** peut s'utiliser sans paramètres, auquel cas **4 octets** sont utilisés pour stocker les valeurs de la colonne. Il est cependant possible de spécifier une précision et une échelle, de la même manière que pour DECIMAL et NUMERIC.

Quant à **REAL** (**4 octets**) et **DOUBLE** (**8 octets**), ils ne supportent pas de paramètres. MySQL qui utilise 8 octets dans les deux cas : conseille d'utiliser DOUBLE pour éviter les surprises en cas de changement de SGBDR.

- **VALEURS EXACTES VS VALEURS APPROCHEES**

Les nombres stockés en tant que **NUMERIC** ou **DECIMAL** sont stockés sous forme de chaînes de caractères. Par conséquent, c'est la valeur exacte qui est stockée. Cependant, les types **FLOAT**, **DOUBLE** et **REAL** sont stockés sous forme de nombres, et **c'est une valeur approchée qui est stockée**.

Cela signifie que si vous stockez par exemple 56,6789 dans une colonne de type FLOAT, en réalité, MySQL stockera une valeur qui se rapproche de 56,6789 (par exemple, 56,678900000000000001). Cela peut poser un problème pour des comparaisons notamment (56,678900000000000001 n'étant pas égal à 56,6789).

S'il est nécessaire de conserver la précision exacte de vos données (*l'exemple type est celui des données bancaires*), il est donc conseillé d'utiliser un type numérique à valeur exacte (NUMERIC ou DECIMAL).

b. Types alphanumériques

i. Chaînes de type texte : CHAR et VARCHAR

Pour stocker un texte relativement court (moins de 255 caractères), vous pouvez utiliser les types CHAR et VARCHAR. Ces deux types s'utilisent avec un paramètre qui précise la taille que peut prendre votre texte (entre 1 et 255).

La différence entre CHAR et VARCHAR est la manière dont ils sont stockés en mémoire. Un CHAR(x) stockera toujours x caractères, en remplissant si nécessaire le texte avec des

espaces vides pour le compléter, tandis qu'un VARCHAR(x) stockera jusqu'à x caractères (entre 0 et x), et stockera en plus en mémoire la taille du texte stocké.

Petit tableau explicatif, en prenant l'exemple d'un CHAR ou d'un VARCHAR de 5 caractères au maximum :

Texte	CHAR (5)	Mémoire requise	VARCHAR (5)	Mémoire requise
''	' '	5 octets	''	1 octet
'tex'	'tex '	5 octets	'tex'	4 octets
'texte'	'texte'	5 octets	'texte'	6 octets
'texte trop long'	'texte'	5 octets	'texte'	6 octets

REMARQUE
Dans le cas où le texte fait la longueur maximale autorisée, un CHAR(x) prend moins de place en mémoire qu'un VARCHAR(x). Préférez donc le CHAR(x) dans le cas où vous savez que vous aurez toujours x caractères. Cependant, si la longueur de votre texte risque de varier d'une ligne à l'autre, définissez votre colonne comme un VARCHAR(x).

Dans le tableau ci-dessus, tous les caractères sont des caractères simples, sans accents. Dans un texte comprenant des accents et encodé en UTF-8, les caractères accentués occupent deux octets en mémoire. Il est donc possible, pour un CHAR (5), d'occuper plus de 5 octets en mémoire (*mais impossible d'y stocker plus que 5 caractères*).

ii. Chaînes de type texte : TEXT

Pouvoir stocker des textes de plus de 255 caractères, Il suffit d'utiliser le type **TEXT**, ou un de ses dérivés **TINYTEXT**, **MEDIUMTEXT** ou **LONGTEXT**. La différence entre ceux-ci est la place qu'ils permettent d'occuper en mémoire. Petit tableau habituel :

Type	Mémoire occupée	Longueur maximale
TINYTEXT	2828 octets	Longueur de la chaîne + 1 octet
TEXT	216216 octets	Longueur de la chaîne + 2 octets
MEDIUMTEXT	224224 octets	Longueur de la chaîne + 3 octets
LONGTEXT	232232 octets	Longueur de la chaîne + 4 octets

iii. Chaînes de type binaire

Les types binaires sont parfaits pour stocker des données "brutes" comme des images par exemple, tandis que les chaînes de texte sont parfaites pour stocker du texte.

Les types binaires **VARBINARY(x)**et **BINARY(x)** sont définis de la même façon que les types de chaînes de texte.

Pour les chaînes plus longues, il existe les types **TINYBLOB**, **BLOB**, **MEDIUMBLOB** et **LONGBLOB**, avec les mêmes limites de stockage que les types TEXT.

c. Types propres à MySQL : SET et ENUM

i. ENUM

Une colonne de type **ENUM** est une colonne pour laquelle on définit un certain nombre de valeurs autorisées qui serviront à alimenter une liste de choix, de type "chaîne de caractères". **ENUM** peut avoir au maximum **65535** valeurs possibles.

- si vous essayez d'introduire une chaîne non autorisée, MySQL stockera une **chaîne vide** (**''**) dans le champ ;
- si vous autorisez le champ à ne pas contenir de valeur le champ contiendra **NULL**, qui signifie aucune donnée n'est renseignée. Ceci correspond à "pas de valeur" en SQL.

Exemple :

```
marque ENUM('toyota', 'benz', 'kia')
```

La colonne **marque** ne pourra que contenir les chaînes "toyota", "benz" ou "kia".

ii. SET :

Une colonne **SET** est une colonne qui permet de stocker une chaîne de caractères dont les valeurs possibles sont prédéfinies. La différence avec ENUM, c'est que l'on peut stocker dans la colonne entre 0 et x valeur(s), x étant le nombre de valeurs autorisées.

Exemple :

```
marque SET('toyota', 'benz', 'kia')
```

On pourra stocker par exemple dans cette colonne :

- ' ' (chaîne vide) ;
- 'toyota' ;
- 'toyota,kia' ;
- 'toyota,kia,benz' ;
- 'kia,benz' ;
- …

NB. On ne peut pas stocker la même valeur plusieurs fois dans un SET.

d. Types temporels

Pour les données temporelles, MySQL dispose de cinq types qui permettent, lorsqu'ils sont bien utilisés, de faire énormément de choses.

Lorsque vous stockez une date dans MySQL, certaines vérifications sont faites sur la validité de la date entrée (*le jour doit être compris entre 1 et 31 et le mois entre 1 et 12 par exemple*).
Les cinq types temporels de MySQL sont **DATE**, **DATETIME**, **TIME**, **TIMESTAMP** et **YEAR**.

i. DATE, TIME et DATETIME

Comme son nom l'indique, **DATE** sert à stocker une **date**. **TIME** sert quant à lui à stocker une **heure**, et **DATETIME** stocke une **date** et une **heure**.

ii. DATE

Pour entrer une date, l'ordre des données est la seule contrainte. Il faut donner d'abord l'année (*deux ou quatre chiffres*), ensuite le mois (*deux chiffres*), et, pour finir, le jour (*deux chiffres*), sous forme de nombre ou de chaîne de caractères. S'il s'agit d'une chaîne de caractères, n'importe quelle ponctuation peut être utilisée pour délimiter les parties (ou aucune).

Exemples d'expressions correctes (*AAAA=Années, MM=Mois et JJ= Jours*) :

- 'AAAA-MM-JJ' (*c'est sous ce format-ci qu'une DATE est stockée dans MySQL*)
- 'AAMMJJ' ; 'AAAA/MM/JJ' ; 'AA+MM+JJ' ; 'AAAA%MM%JJ'
- AAAAMMJJ (*nombre*) et AAMMJJ (*nombre*)

iii. DATETIME

Permet de stocker une heure, en plus d'une date. Pour entrer un DATETIME, c'est le même principe que pour DATE. Quelques exemples corrects (*HH=Heures, MM=Minutes et SS=Secondes*) :

- 'AAAA-MM-JJ HH :MM : SS' (c'est sous ce format-ci qu'un DATETIME est stocké dans MySQL)
- 'AA*MM*JJ HH+MM+SS' et AAAAMMJJHHMMSS (nombre)

iv. TIME

Permet non seulement de stocker une heure précise, mais aussi un intervalle de temps. On n'est donc pas limité à 24 heures, et il est même possible de stocker un nombre de jours ou un intervalle négatif. Comme dans DATETIME, il faut d'abord donner l'heure, puis les minutes, puis les secondes, chaque partie pouvant être séparée des autres par le caractère « : ».

Exemples :

- 'HH:MM:SS' ; 'HHH:MM:SS' ; 'MM:SS' ; 'J HH:MM:SS' ; 'HHMMSS'
- HHMMSS (*nombre*)

v. YEAR

Permet de retenir que l'année et ne prend qu'un seul octet en mémoire.

vi. TIMESTAMP

Stocké sur 4 octets, Timestamp d'une date est le nombre de secondes écoulées depuis le 1er janvier 1970, 0 h 0 min 0 s (TUC) et la date en question (*il existe une limite supérieure : le 19 janvier 2038 à 3 h 14 min 7 s*).

C. EXECUTION DES COMMANDES SQL

1. Création et suppression d'une base de données

a. CREATION

La commande SQL pour créer une base de données est la suivante :

```
CREATE DATABASE nom_base;
```

Lors de la création des tables dans la base de données, celles-ci seront automatiquement encodées en UTF-8.

Toutefois, vous trouverez ci-dessous la commande complète permettant de créer votre première base de données. Merci de ne pas l'exécuter pour le moment.

```
CREATE DATABASE cdpdb CHARACTER SET 'utf8';
```

b. SUPPRESSION

```
DROP DATABASE nom_base;
```

Si vous essayez cette commande alors que la base de données n'existe pas, MySQL vous affichera une erreur :

```
mysql> DROP DATABASE cdpdb;
ERROR 1008 (HY000) : Can't drop database cdpdb; database doesn't exist
mysql>
```

Pour éviter les messages d'erreur lors de la suppression d'une base de données inexistante, il est conseillé d'utiliser l'option **IF EXISTS**. Cette option permet à MySQL de vérifier l'existence de la base de données avant d'exécuter la commande de suppression. Si la base de données n'existe pas, aucune erreur ne sera affichée et la commande sera simplement ignorée.

La syntaxe est la suivante :

```
DROP DATABASE IF EXISTS nom_de_la_base;
```

Par exemple :

```
DROP DATABASE IF EXISTS cdpdb;
```

Dans cet exemple, MySQL vérifie si une base de données nommée cdpdb existe. Si elle est présente, elle sera supprimée. Sinon, la commande se termine sans provoquer d'erreur.

L'utilisation de IF EXISTS est particulièrement utile dans les scénarios suivants :

- Lors de l'automatisation de scripts SQL où l'existence de la base de données n'est pas garantie.
- Dans les environnements de développement ou de test où les bases de données peuvent être recréées ou modifiées fréquemment.
- Pour éviter de stopper l'exécution de scripts à cause d'une erreur liée à une base de données inexistante.

c. UTILISATION D'UNE BASE DE DONNEES

Il est désormais temps de procéder à la création de votre base de données **cdpdb** à l'aide de la commande suivante :

```
CREATE DATABASE cdpdb CHARACTER SET 'utf8';
```

Pour pouvoir agir sur une base de données créé, vous devez d'abord la sélectionner. Une fois de plus, la commande est très simple :

```
USE cdpdb ;
```

```
mysql> drop database if exists cdpdb;
Query OK, 0 rows affected (0.02 sec)

mysql> create database cdpdb;
Query OK, 1 row affected (0.00 sec)

mysql> use cdpdb;
Database changed
mysql>
```

2. MANIPULATION DES TABLES

a. TYPE DE COLONNE

Avant de choisir le type des colonnes pour votre table, il faut choisir les colonnes que l'on va définir. Dans le cadre de ce cours nous allons utiliser une table comme définie dans le tableau suivant. Nous allons donc créer une table Voiture comportant les attributs suivants :

Caractéristique	Nom de la colonne	Champ	Null ?
Numéro d'identification	Id	SMALLINT	Non
Immatriculation	Immat	VARCHAR(8)	non
Marque	Marque	VARCHAR(15)	non
Type de moteur	Moteur	VARCHAR(10)	non
Date mise en circulation	date_circ	DATETIME	non
Kilométrage	Km	SMALLINT	non
Prix d'achat	PxAchat	INT	oui
Observation	Commentaires	TEXT	oui

i. CLES PRIMAIRES

La Clé primaire d'une table est une contrainte d'unicité.

Composée d'une ou plusieurs colonnes, la Clé primaire d'une ligne permet d'identifier de manière unique cette ligne dans la table. Par exemple, dans le cas où vous essayez d'insérer un clé primaire qui existe déjà, vous obtiendrez avec MySQL l'erreur suivante :

```
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
```

Pour définir une clé primaire on utilise le mot-clé PRIMARY KEY.

Nous définirons donc dans le cadre de notre table Voiture la colonne « id » comme la clé primaire de la table, de la manière suivante PRIMARY KEY(id).

ii. AUTO-INCREMENTATION

Si l'on déclare qu'une colonne doit s'auto-incrémenter (*grâce au mot-clé AUTO_INCREMENT*), MySQL va chercher la valeur à mettre lors de la prochaine insertion, en prenant la dernière valeur insérée dans cette colonne et en l'incrémentant par défaut de 1.

Dans notre cas, on peut décider pour chaque Voiture, d'une valeur pour id. Le plus simple, et le plus logique, est de donner le numéro 1 à la première voiture enregistrée, puis le numéro 2 au second, etc. d'où une auto-incrementation.

b. MOTEURS DE TABLES

Les moteurs de tables sont une spécificité de MySQL. Ce sont des moteurs de stockage. Cela permet de gérer différemment les tables selon l'utilité que l'on en a. Les deux moteurs les plus connus sont MyISAM et InnoDB.

- MyISAM : C'est le moteur par défaut. Les commandes d'insertion et sélection de données sont particulièrement rapides sur les tables utilisant ce moteur. Cependant, il ne gère pas certaines fonctionnalités importantes comme les clés étrangères, qui permettent de vérifier l'intégrité d'une référence d'une table à une autre table ou les transactions, qui permettent de réaliser des séries de modifications "en bloc", ou au contraire d'annuler ces modifications.
- InnoDB : Plus lent et plus gourmand en ressources que MyISAM, ce moteur gère les clés étrangères et les transactions.

  De plus, en cas de crash du serveur, il possède un système de récupération automatique des données.

c. SYNTAXE DE CREATION DE TABLE

Avant de voir la syntaxe permettant de créer une table, résumons un peu. Nous voulons donc créer une table VOITURE avec six colonnes telles que décrites dans le tableau suivant.

CARACTERISTIQUE	NOM DU CHAMP	TYPE	NULL ?	DIVERS
Numéro d'identification	Id	SMALLINT	non	Clé primaire + auto-incrémente +UNSIGNED
Immatriculation	Immat	VARCHAR(8)	non	-
Marque	Marque	VARCHAR(15)	non	-
Type de moteur	Moteur	VARCHAR(10)	non	-
Date mise en circulation	date_circ	DATETIME	non	-
Kilométrage	Km	INT	non	-
Prix d'achat	PxAchat	INT	oui	-
Observation	Commentaires	TEXT	oui	-

i. Syntaxe

Par souci de clarté, je vais diviser l'explication de la syntaxe de CREATE TABLE en deux. La première partie vous donne la syntaxe globale de la commande, et la deuxième partie s'attarde sur la description des colonnes créées dans la table.

- **Création de la table**

```
CREATE TABLE [IF NOT EXISTS] Nom_table (
    colonne1 description_colonne1,
    [colonne2 description_colonne2,
    colonne3 description_colonne3,
    [...,]
    [PRIMARY KEY (colonne_clé_primaire)]
)
[ENGINE=moteur];
```

Le « IF NOT EXISTS » est facultatif.

Application : création de la table VOITURE

```
mysql>  create table Voiture(
   id smallint unsigned not null auto_increment,
   immat varchar(10) not null,
   marque varchar(15) not null,
   moteur varchar(10) not null,
   datecirc datetime null,
   km smallint not null,
   pxachat int not null,
   comment text null,
   primary key(id)
    )
   engine=innodb;
Query OK, 0 rows affected (0.03 sec)
```

- **Vérifications :** voici deux commandes permettant de vérifier que vous avez bien créé la table voiture.

```
SHOW TABLES ; -- liste toutes les tables d'une base de données

DESCRIBE voiture ; -- liste les colonnes de la table voiture avec leurs
caractéristiques
```

```
mysql> show tables;
+-----------------+
| Tables_in_cdpdb |
+-----------------+
| voiture         |
+-----------------+
1 row in set (0.01 sec)

mysql> describe voiture;
+----------+--------------------+------+-----+---------+----------------+
| Field    | Type               | Null | Key | Default | Extra          |
+----------+--------------------+------+-----+---------+----------------+
| id       | smallint unsigned  | NO   | PRI | NULL    | auto_increment |
| immat    | varchar(10)        | NO   |     | NULL    |                |
| marque   | varchar(15)        | NO   |     | NULL    |                |
| moteur   | varchar(10)        | NO   |     | NULL    |                |
| datecirc | datetime           | YES  |     | NULL    |                |
| km       | smallint           | NO   |     | NULL    |                |
| pxachat  | int                | NO   |     | NULL    |                |
| comment  | text               | YES  |     | NULL    |                |
+----------+--------------------+------+-----+---------+----------------+
8 rows in set (0.01 sec)
```

d. SUPPRESSION D'UNE TABLE

La commande pour supprimer une table est la même que celle pour supprimer une base de données. Elle est, bien sûr, à utiliser avec prudence, car irréversible.

```
DROP TABLE voiture;
```

e. MODIFICATION D'UNE TABLE

Les requêtes permettant de modifier une table porteront sur la modification des colonnes d'une table (*ajout, modification et suppression de colonnes*).

Dans les trois cas, c'est la commande ALTER TABLE qui sera utilisée, une variante existant pour chacune des opérations :

```
ALTER TABLE nom_table ADD ... -- permet d'ajouter quelque chose (une colonne
par exemple)

ALTER TABLE nom_table DROP ... -- permet de retirer quelque chose
ALTER TABLE nom_table CHANGE ...

ALTER TABLE nom_table MODIFY ... -- permettent de modifier une colonne
```

i. Ajout de colonne

On utilise la syntaxe suivante :

```
ALTER TABLE nom_table
ADD [COLUMN] nom_colonne description_colonne;
```

[COLUMN] : est facultatif, donc, si à la suite de ADD , vous ne précisez pas ce que vous voulez ajouter, MySQL considérera qu'il s'agit d'une colonne.

nom_colonne : correspond au nom de la nouvelle colonne à ajouter
description_colonne : correspond à la même chose que lorsque l'on crée une table.

Exemple :
Ajoutons la colonne « modèle » à la table voiture. Il s'agit d'une colonne de type VARCHAR. Cette colonne peut être ne peut être NULL. Cela donne :

```
ALTER TABLE voiture


ADD COLUMN modele VARCHAR(10);
```

« DESCRIBE voiture ; » vous permettra de vérifier les changements apportés.

```
mysql> alter table voiture
    -> add column modele varchar(10) not null;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> describe voiture;
+----------+-------------------+------+-----+---------+----------------+
| Field    | Type              | Null | Key | Default | Extra          |
+----------+-------------------+------+-----+---------+----------------+
| id       | smallint unsigned | NO   | PRI | NULL    | auto_increment |
| immat    | varchar(10)       | NO   |     | NULL    |                |
| marque   | varchar(15)       | NO   |     | NULL    |                |
| moteur   | varchar(10)       | NO   |     | NULL    |                |
| datecirc | datetime          | YES  |     | NULL    |                |
| km       | smallint          | NO   |     | NULL    |                |
| pxachat  | int               | NO   |     | NULL    |                |
| comment  | text              | YES  |     | NULL    |                |
| modele   | varchar(10)       | NO   |     | NULL    |                |
+----------+-------------------+------+-----+---------+----------------+
9 rows in set (0.00 sec)
```

ii. Suppression de colonne

```
ALTER TABLE nom_table
DROP [COLUMN] nom_colonne;
```

Exemple :
Supprimer la colonne modele, que vous venez d'ajouter à la table voiture.

```
ALTER TABLE voiture

DROP COLUMN modele; -- Suppression de la colonne modele
```

iii. Modification de colonne

Pour commencer, ajouter la colonne « place » à la table voiture comme ci-dessous

```
ALTER TABLE voiture
 ADD COLUMN place SMALLINT NOT NULL;
```

- **Changement du nom de la colonne**

Vous pouvez utiliser la commande suivante pour changer le nom d'une colonne :

```
ALTER TABLE nom_table

CHANGE ancien_nom nouveau_nom description_colonne;
```

Exemple :
Renommer la colonne « place » en « Nbreplace »

```
ALTER TABLE voiture
CHANGE place Nbreplace SMALLINT NOT NULL;
```

- **Changement du type de données**

Les mots-clés CHANGE et MODIFY peuvent être utilisés pour changer le type de donnée de la colonne, mais aussi changer la valeur par défaut ou ajouter/supprimer une propriété AUTO_INCREMENT. Si vous utilisez CHANGE, vous pouvez, comme on vient de le voir, renommer la colonne en même temps. Si vous ne désirez pas la renommer, il suffit d'indiquer deux fois le même nom.

Voici les syntaxes possibles :

```
ALTER TABLE nom_table

CHANGE ancien_nom nouveau_nom nouvelle_description;

ALTER TABLE nom_table

MODIFY nom_colonne nouvelle_description;
```

Des exemples pour illustrer cela :

```
ALTER TABLE voiture

CHANGE moteur consommation VARCHAR(20) NOT NULL; -- Changement du nom et de la
taille de la colonne existante
ALTER TABLE voiture

CHANGE id id BIGINT NOT NULL; -- Changement du type sans renommer
```

```
ALTER TABLE voiture

MODIFY id BIGINT NOT NULL AUTO_INCREMENT; -- Ajout de l'auto-incrémentation

ALTER TABLE voiture

MODIFY consommation VARCHAR(20) NOT NULL DEFAULT 'ESSENCE';
-- Changement de la description (même type mais ajout d'une valeur par défaut)
```

NB : Utilisez à chaque fois la commande DESCRIBE (« DESCRIBE Voiture ») pour vérifier les changements opérés sur la table Voiture.

f. INSERTION DANS UNE TABLE

Ce chapitre est consacré à l'insertion de données dans une table. Rien de bien compliqué, mais c'est évidemment crucial. En effet, que serait une base de données sans données ?

Nous verrons entre autres :

- comment insérer une ligne dans une table ;
- comment insérer plusieurs lignes dans une table ;
- comment exécuter des requêtes SQL écrites dans un fichier (requêtes d'insertion ou autres) ;
- comment insérer dans une table des lignes définies dans un fichier de format particulier.

i. Syntaxe de INSERT

Pour insérer une ligne dans une table, nous avons deux possibilités : soit donner une valeur pour chaque colonne de la ligne, soit ne donner les valeurs que de certaines colonnes, auquel cas il faut bien sûr préciser de quelles colonnes il s'agit.

- **Insertion sans préciser les colonnes**

```
INSERT INTO nom_table

VALUES (valeur1, valeur2, valeur3,…, valeurn);
```

Exemples :
Insertion de quelques lignes dans la table voiture sans renseigner les colonnes.

```
INSERT INTO voiture

VALUES (1,'TG-4541-AC', 'BENZ', 'HYBRIDE', '2022-01-21 08:00:00', 21,45000000, 'Couleur claire ',5);
```

```
INSERT INTO voiture

VALUES (2,'TG-8969-AK', 'SUZUKI', 'ESSENCE', '2022-12-20 1:00:00', 0,12000000, ' ',5);
```

Autres exemples avec des valeurs NULL :

```
INSERT INTO voiture

VALUES (1,'TG-0541-AY', 'TOYOTA', 'DIESEL', NULL , 5000,9000000, NULL,7);
INSERT INTO voiture

VALUES (NULL,'TG-4541-AC', 'BENZ', 'HYBRIDE', '2022-01-21 08:00:00', 21,45000000,
'Couleur claire ',5);
```

NB: on donne NULL comme valeur d'id, ce qui en principe est impossible puisque id est défini comme NOT NULL et comme clé primaire. Cependant, l'auto-incrémentation fait que MySQL va calculer la valeur qu'il faut donner à la ligne.

- **Insertion avec précision des colonnes**

```
INSERT INTO nom_table (Colonne1, colonne2, colonne3, …, Colonnen)

VALUES (valeur1, valeur2, valeur3,…, valeurn);
```

Exemples :

```
INSERT INTO voiture (Immat,marque,moteur,datecirc, km, pxachat, comment,
Nbreplace)

VALUES ('TG-7841-BI', 'TOYOTA', 'DIESEL', NULL , 1000,7000000, 'couleur verte,5);
```

```
INSERT INTO voiture (Immat,marque,moteur,km, pxachat, Nbreplace)

VALUES ('TG-0001-AF', 'TOYOTA', 'DIESEL',8500,4500000,5);
```

- **Syntaxe alternative de MySQL**

MySQL propose une syntaxe alternative à INSERT INTO dont voici la syntaxe

```
INSERT INTO voiture
SET Immat='TG-9696-AH', marque='Hyundai', moteur='DIESEL',km=25000,
pxAchat=80000000,Nbreplace=5;
```

- **Insertion multiple**

Si vous avez plusieurs lignes à introduire, il est possible de le faire en une seule requête de la manière suivante :

```
INSERT INTO voiture (Immat,marque,moteur,km, pxachat, Nbreplace)
VALUES ('TG-7854-AM', 'BENZ', 'DIESEL',500,6000000,5),
       ('TG-3525-BA', 'TOYOTA', 'ELECTRIQUE',0,1500000,7),
       ('TG-0789-AK', 'SUZIKI', 'ESSENCE',0,1450000,5);
```

Pour vérifier, vous pouvez utiliser la requête suivante :

```
SELECT * FROM voiture;
```

➢ Utilisation de fichiers externes

Pour éviter d'écrire toutes les requêtes d'insertion une à une, il est possible d'utiliser un fichier texte pour interagir avec la base de données créée.

Il s'agit « d'Exécuter des commandes SQL à partir d'un fichier » ou à partir « des données à formatées dans un fichier » (*un fichier csv par exemple*).

La solution consiste à écrire les requêtes dans un fichier texte, puis de dire à MySQL d'exécuter les requêtes contenues dans ce fichier. Et pour lui dire cela, voici la commande :

```
SOURCE monFichier.sql;
```

Ou

```
\. monFichier.sql;
```

Exemple : utiliser le chemin complet vers le fichier

```
SOURCE D:/Regular/Private/SKOOL/CDP/BDD/DATA.sql
```

➢ Remplissage de la table Voiture

Nous allons reprendre les choses proprement pour avoir les mêmes données.
Pour ce faire nous allons :

- Supprimer la table Voiture existante
- Créer une nouvelle base de données dénommée « PARC_AUTO »
- Créer une table Voiture
- Constituer un fichier de données « data_voitures.sql»
- Utilisez la commande « Source » pour insérer dans la table crée les données contenues dans le fichier fourni (*data_voitures.sql*).

```
mysql> create database PARC_AUTO;
Query OK, 1 row affected (0.01 sec)

mysql> use PARC_AUTO;
Database changed
mysql> create table Voiture(
    ->      id smallint unsigned not null auto_increment,
    ->      immat varchar(10) null,
    ->      marque varchar(15) not null,
    ->      moteur varchar(10) not null,
    ->      datecirc datetime null,
    ->      km smallint not null,
    ->      pxachat int not null,
    ->      nbreplace tinyint not null,
    ->      primary key(id)
    ->      )
    ->      engine=innodb;
Query OK, 0 rows affected (0.02 sec)

mysql> describe voiture;
+-----------+------------------+------+-----+---------+----------------+
| Field     | Type             | Null | Key | Default | Extra          |
+-----------+------------------+------+-----+---------+----------------+
| id        | smallint unsigned | NO   | PRI | NULL    | auto_increment |
| immat     | varchar(10)      | YES  |     | NULL    |                |
| marque    | varchar(15)      | NO   |     | NULL    |                |
| moteur    | varchar(10)      | NO   |     | NULL    |                |
| datecirc  | datetime         | YES  |     | NULL    |                |
| km        | smallint         | NO   |     | NULL    |                |
| pxachat   | int              | NO   |     | NULL    |                |
| nbreplace | tinyint          | NO   |     | NULL    |                |
+-----------+------------------+------+-----+---------+----------------+
8 rows in set (0.00 sec)
```

Mettez à jour la table voiture que vous venez de créer en ajoutant le champ « nbreplace » de type **tinyint**, qui doit être **obligatoire**.

Exécutez la commande suivante pour insérer les enregistrements dans la table voiture à partir du fichier source C:/SQL/data_voitures.sql, en vous assurant que ce fichier se trouve bien sur votre ordinateur.

```
SOURCE C:/SQL/data_voitures.sql
```

Vérifier à l'aide de la commande `SELECT * FROM voiture;` le contenu de la table Voiture.

3. SELECTION DES DONNEES D'UNE TABLE

a. *COMMANDE SELECT*

La commande qui permet de sélectionner et d'afficher des données s'appelle SELECT.
SELECT permet donc d'afficher des données directement : des chaînes de caractères, des résultats de calculs, etc.

Exemple :

```
SELECT 'BIENVENUE AU COURS DE DASE DE DONNEES AU CDP!';

SELECT 45+40;
```

SELECT permet également de sélectionner des données à partir d'une table. Pour cela, il faut ajouter la clause FROM à la commande SELECT.
La clause FROM définit de quelle structure (*dans notre cas, une table*) viennent les données.

Voici la syntaxe :

```
SELECT colonne1, colonne2, ...

FROM nom_table;
```

Par exemple, si l'on veut sélectionner l'immatriculation, la marque, et le prix d'achat des voitures présentes dans la table Voiture, on écrira la requête suivante :

```
SELECT immat, marque, pxachat

FROM Voiture;
```

Pour sélectionner toutes les colonnes de la table voiture, on pourra utiliser le caractère « * » dans la requête :

```
SELECT *

FROM Voiture;
```

b. *CLAUSE WHERE*

Les requêtes SELECT ci-dessus sélectionnent toutes les lignes de la table. Or, bien souvent, on ne veut qu'une partie des données.

La clause WHERE permet à la requête SELECT permet de restreindre les résultats selon des critères de recherche.

Exemple : *Voici la requête permettant de sélectionner les voitures de marque « KIA » de la table Voiture.*

```
SELECT *
 FROM Voiture
 WHERE marque='KIA';
```

Comme « KIA » est une chaîne de caractères, on doit bien sûr l'entourer de guillemets.

i. OPERATEURS DE COMPARAISON

Les opérateurs de comparaison

Les opérateurs de comparaison sont les symboles que l'on utilise pour définir les critères de recherche (le = dans notre exemple précédent). Huit opérateurs simples peuvent être utilisés.

Opérateur	Signification
=	Egal
<	Inférieur
<=	Inférieur ou égal
>	Supérieur
>=	Supérieur ou égal
<> ou !=	Différent
<=>	Egal (valable pour NULL aussi)

Exemples :

```
SELECT *
FROM Voiture
WHERE datecirc < '2010-01-01'; -- Voitures mises en circulation avant 2010

SELECT *
FROM Voiture
WHERE marque <> 'Hyundai'; -- Toutes les voitures sauf les Hyundai
```

ii. COMBINAISONS DE CRITERES

On peut combiner des critères de sélection dans la clause WHERE avec des opérateurs logiques, qui sont au nombre de quatre :

Opérateur	Symbole	Signification
AND	&&	ET
OR	||	OU
XOR		OU exclusif
NOT	!	NON

Exemples

AND : Sélectionner toutes les voitures KIA à ESSENCE. On veut sélectionner les voitures qui sont à la fois des KIA ET ESSENCE. On utilisera l'opérateur AND :

```
SELECT *
```

```
FROM Voiture
WHERE marque='KIA' AND consommation='Essence';

-- OU

SELECT *
FROM Voiture
WHERE marque='KIA' && consommation='Essence';
```

OR : Sélectionner les KIA ou les NISSAN. Je désire donc obtenir les voitures qui sont des KIA OU des NISSAN :

```
SELECT *
FROM Voiture
WHERE marque='KIA' OR marque='NISSAN';

-- OU

SELECT *
FROM Voiture
WHERE marque='KIA' || marque='NISSAN';
```

Remarque :

Il est conseillé d'utiliser plutôt « OR » que « || », car, dans la majorité des SGBDR (et dans la norme SQL), l'opérateur || sert à la concaténation, c'est-à-dire à rassembler plusieurs chaînes de caractères en une seule. Il vaut donc mieux prendre l'habitude d'utiliser OR, au cas où vous changeriez un jour de SGBDR.

NOT : Sélectionner toutes les voitures KIA sauf celles de moteur à ESSENCE :

```
SELECT *
FROM Voiture
WHERE marque='KIA' AND NOT consommation='Essence';

-- OU

SELECT *
FROM Voiture
WHERE marque='KIA' AND ! (moteur='Essence');
```

Les parenthèses sont indispensables dans le cas du « ! » , sinon il porte uniquement sur moteur.

XOR : Sélectionner les voitures qui sont soit à essence, soit des KIA (mais pas les deux) :

```
SELECT *
FROM Voiture
WHERE moteur='Essence' XOR marque='KIA';
```

iii. SELECTION COMPLEXE

Lorsque plusieurs critères sont utilisés dans une requête et que plusieurs opérateurs logiques doivent être donc combinés, il est extrêmement important de bien structurer la requête, en particulier, il faut placer des parenthèses au bon endroit.

Exemple simple :

Critères : « rouge AND vert OR bleu » Qu'accepte-ton ?

- *Ce qui est rouge et vert, et ce qui est bleu ?*
  Ou
- *Ce qui est rouge, et soit vert, soit bleu ?*

Il faut donc mettre les parenthèses pour éviter de se tromper sur ce que l'on désire sélectionner.

Exercice 1

Sélectionner les voitures qui sont soit mis en circulation après 2009, soit des TOYOTA essences ou hybrides, mais dans le cas des hybrides, elles doivent être mises en circulation avant juin 2007.

Solution

L'astuce pour résoudre ce problème, c'est de penser en niveaux. On va donc découper la requête.

On cherche donc :

- les voitures mises en circulation après 2009 ;
- les TOYOTA essences et hybrides (uniquement mises en circulation avant juin 2007 pour les hybrides). C'est mon premier niveau. L'opérateur logique sera OR puisqu'il faut que les voitures répondent à un seul des deux critères pour être sélectionnées.

On continue à découper. Le premier critère ne peut plus être subdivisé, contrairement au deuxième. Je cherche :

- les voitures mises en circulation après 2009
- et les TOYOTA :
  - essences ;
  - et hybrides mises en circulation avant juin 2007.

Voilà, vous avez défini les différents niveaux, il n'y a plus qu'à écrire la requête avec les bons opérateurs logiques.

```
SELECT *
FROM voiture
WHERE datecirc > '2009-12-31'
    OR
    ( marque='TOYOTA'
         AND
        ( moteur='essence'
             OR ( moteur='hybride' AND datecirc < '2007-06-01' )
        )
    ) ;
```

Exercice 2

Sélectionner les voitures mises en circulation après 2020, des TOYOTA essences ou hybrides, mais dans le cas des hybrides, elles doivent être mises en circulation avant juin 2009.

iv. CAS DE NULL

Le marqueur NULL (qui représente donc "pas de valeur") est un peu particulier. Vous ne pouvez pas tester directement « colonne = NULL » ou « colonne <> NULL ». En fait, elles ne renvoient aucun résultat. C'est donc ici qu'intervient notre opérateur de comparaison un peu spécial « <=> » qui permet de reconnaître NULL. Une autre possibilité est d'utiliser les mots-clés IS NULL, et si l'on veut exclure les NULL : IS NOT NULL.

Exemples :

```
SELECT *
FROM voiture
WHERE datecirc <=> NULL; -- sélection des voitures qui ne sont pas mises en circulation


-- OU


SELECT *
FROM voiture
WHERE datecirc IS NULL;


SELECT *
FROM voiture
WHERE immat IS NOT NULL;
-- sélection des voitures nom immatriculées
```

v. TRI DES DONNEES

Lorsque vous faites un SELECT, les données sont récupérées dans un ordre défini par MySQL ou par votre SGBD, mais qui n'a aucun sens pour vous. Or, bien souvent, vous voudrez trier à votre manière. Par date de mise en circulation par exemple, ou bien par marque, ou par immatriculation, etc.

Pour trier vos données, il suffit d'ajouter ORDER BY « colonne de tri » à votre requête après les critères de sélection de WHERE (s'il y en a). Bien entendu « colonne de tri » désigne la colonne sur laquelle vous voulez trier vos données.

Exemple : Sélectionner toutes les TOYOYA triées par date de mise en circulation.

```
SELECT *
FROM voiture
WHERE marque='TOYOTA'  ORDER BY datecirc;
```

- **Tri ascendant ou descendant**

Les tris effectués peuvent être Ascendants (*plus petit au plus grand*) ou Descendants (*plus grand au plus petit*). Pour déterminer le sens du tri effectué, SQL possède deux mots-clés : ASC pour ascendant, et DESC pour descendant. Par défaut, si vous ne précisez rien, c'est un tri ascendant qui est effectué : du plus petit nombre au plus grand, de la date la plus ancienne à la plus récente, et pour les chaînes de caractères et les textes, c'est l'ordre alphabétique normal qui est utilisé. Si en revanche vous utilisez le mot DESC, l'ordre est inversé : plus grand nombre d'abord, date la plus récente d'abord, et ordre anti-alphabétique pour les caractères.

Pour déterminer le sens du tri effectué, SQL possède deux mots-clés : ASC pour ascendant, et DESC pour descendant.

Cas particulier : les ENUM sont des chaînes de caractères, mais sont triés selon l'ordre dans lequel les possibilités ont été définies. Si par exemple on définit une colonne exemple ENUM('a', 'd', 'c', 'b'), l'ordre ASC sera 'a', 'd', 'c' puis 'b' et l'ordre DESC 'b', 'c', 'd' suivi de 'a'.

Exemple : Sélectionner toutes les TOYOTA par immatriculation décroissante.

```
SELECT *
FROM voiture
WHERE marque='Toyota' AND immat IS NOT NULL
ORDER BY immat DESC;
```

Il est également possible de trier sur plusieurs colonnes.
Par exemple, si vous voulez des résultats triés par marque et, au niveau de chaque marque, triés par date de mise en circulation, il suffit de donner les deux colonnes correspondantes à ORDER BY.

```
SELECT *
FROM Voiture
ORDER BY marque, datecirc;
```

L'ordre dans lequel vous donnez les colonnes est important, le tri se fera d'abord sur la première colonne donnée, puis sur la seconde, etc.
Vous pouvez trier sur autant de colonnes que vous voulez.

vi. ELIMINATION DES DOUBLONS

Il peut arriver que votre requête vous donne plusieurs fois le même résultat tout simplement parce que certaines informations sont présentes plusieurs fois dans la table.

Exemple : vous voulez savoir quelles sont les marques de voiture que vous possédez dans votre parc. la requête est :

```
SELECT marque
FROM voiture;
```

En effet, vous allez bien récupérer toutes les marques de voiture du parc auto, mais si vous avez 500 TOYOTA, vous allez récupérer 500 lignes 'TOYOTA'.
Il y a le mot-clé DISTINCT qui se place juste après SELECT et permet d'éliminer les doublons.

```
SELECT DISTINCT marque
FROM voiture;
```

Attention ! pour éliminer un doublon, il faut que toute la ligne sélectionnée soit égale à une autre ligne du jeu de résultats. Ne seront donc prises en compte que les colonnes que vous avez précisées dans votre SELECT. Uniquement marque donc, dans notre exemple.

vii. RESTREINDRE LES RESULTATS

En plus de restreindre une recherche en lui donnant des critères grâce à la clause WHERE, il est possible de restreindre le nombre de lignes récupérées. Cela se fait grâce à la clause LIMIT.

Syntaxe

LIMIT s'utilise avec deux paramètres

- le nombre de lignes que l'on veut récupérer ;
- le décalage, introduit par le mot-clé OFFSET, qui indique à partir de quelle ligne on récupère les résultats. Ce paramètre est facultatif. S'il n'est pas précisé, il est mis à 0.
-

```
LIMIT nombre_de_lignes [OFFSET decalage];
```

OU
Syntaxe alternative

```
LIMIT [decalage, ]nombre_de_lignes;
```

Exemple :

```
SELECT *
FROM Voiture
LIMIT 10 ;

SELECT *
FROM Voiture  LIMIT 10 OFFSET 5;
```

Avec la première requête, vous devriez obtenir dix lignes, les dix plus petits id, puisque nous n'avons demandé aucun décalage : par défaut OFFSET 0.

id	immat	marque	moteur	datecirc	km	pxachat	nbreplace
1	TG-7854-Z	BENZ	DIESEL	2000-01-21 08:00:00	500	6000000	5
2	TG-6733-AB	BENZ	DIESEL	2002-01-21 18:00:00	0	16000000	5
3	NULL	BENZ	DIESEL	NULL	500	6000000	5
4	TG-2020-AC	BENZ	ESSENCE	2003-03-21 08:00:00	0	9000000	5
5	TG-3733-AC	TOYOTA	HYBRIDE	2010-01-21 18:00:00	0	20000000	7
6	TG-6733-AC	BENZ	DIESEL	2012-01-21 18:00:00	0	9600000	5
7	TG-5420-AF	SUZIKI	ELECTRIC	2013-01-21 18:00:00	10000	21600000	5
8	TG-9420-AF	TOYOYA	ESSENCE	2013-09-21 18:00:00	0	50000000	7
9	TG-1200-AK	SUZIKI	ELECTRIC	2014-01-21 18:00:00	0	11600000	5
10	TG-9420-AK	TOYOYA	ESSENCE	2014-09-21 18:00:00	100	50000000	7

10 rows in set (0.00 sec)

En revanche, dans la deuxième, vous récupérez toujours dix lignes, mais vous devriez commencer au sixième id, puisque l'on a demandé un décalage de cinq lignes.

id	immat	marque	moteur	datecirc	km	pxachat	nbreplace
6	TG-6733-AC	BENZ	DIESEL	2012-01-21 18:00:00	0	9600000	5
7	TG-5420-AF	SUZIKI	ELECTRIC	2013-01-21 18:00:00	10000	21600000	5
8	TG-9420-AF	TOYOYA	ESSENCE	2013-09-21 18:00:00	0	50000000	7
9	TG-1200-AK	SUZIKI	ELECTRIC	2014-01-21 18:00:00	0	11600000	5
10	TG-9420-AK	TOYOYA	ESSENCE	2014-09-21 18:00:00	100	50000000	7
11	TG-0020-AL	TOYOYA	DIESEL	2014-11-21 18:00:00	0	50000000	7
12	TG-4444-AL	HYUNDAI	ESSENCE	2015-09-21 18:00:00	5	10000000	5
13	TG-5420-AL	BMW	ESSENCE	2015-10-21 08:00:00	0	50000000	7
14	NULL	TOYOYA	ESSENCE	NULL	0	50000000	7
15	NULL	BENZ	ESSENCE	NULL	23000	30000000	5

10 rows in set (0.00 sec)

c. ELARGIR LES POSSIBILITES DE LA CLAUSE WHERE

Il arrive que les opérateurs de comparaison classiques (= ; > ; < ; >= ; <= ; <> ; != et <=>) ne soient pas suffisants, en particulier pour des recherches sur des chaînes de caractères. Par exemple, comment faire lorsque l'on cherche toutes les lignes dont le champ "commentaires" contient un mot particulier ?

Nous allons donc revenir plus particulièrement sur la clause WHERE où jusqu'ici, les conditions permises par WHERE étaient très basiques ; alors que cette clause offre bien d'autres possibilités, dont :

- la comparaison avec une valeur incomplète (*chercher les voitures dont la marque commence par une certaine lettre par exemple*) ;
- la comparaison avec un intervalle de valeurs (*entre 2 et 5 par exemple*) ;
- la comparaison avec un ensemble de valeurs (*comparaison avec 5, 6, 10 ou 12, par exemple*).

i. OPERATEURS LIKE

Deux jokers existent pour LIKE :

- '%' : qui représente n'importe quelle chaîne de caractères, quelle que soit sa longueur (y compris une chaîne de longueur 0) ;
- '_' : qui représente un seul caractère.

Quelques exemples :

- 'c%' cherchera toutes les chaînes de caractères commençant par 'c'.
- 'c_' : toutes les chaînes de caractères contenant deux lettres dont la première est 'c'
- '%ch%ne' : toutes les chaînes de caractères contenant 'ch' et finissant par 'ne'
- 'p_rl_' cherchera toutes les chaînes de caractères commençant par un 'p' suivi d'un caractère, puis de "rl" et enfin se terminant par un caractère.

Remarque

Il faut donc signaler à MySQL que vous ne désirez pas utiliser `%` ou `_` en tant que joker, mais bien en tant que caractère de recherche. Pour ça, il suffit de mettre le caractère d'échappement `\`, dont je vous ai déjà parlé, devant le `%` ou le `_`.

Exemple:

Liste des voiture de la AB.

```
SELECT *
FROM voiture
WHERE immat LIKE '%AB'; -- ou immat LIKE '__-____-AB'
```

Liste des véhicules mises en sieur au cours d'un mois de Février

```
SELECT *
FROM voiture
WHERE immat LIKE '%02%';
```

➢ *EXCLUSION DES CHAINES DE CARACTERES*

L'opérateur logique NOT est utilisable avec LIKE. Si l'on veut rechercher les voitures dont la série n'est pas AB, on peut donc écrire :

```
SELECT *
FROM Voiture
WHERE immat NOT LIKE '%AB';
```

➢ *SENSIBILITE A LA CASSE*

LIKE 'chaîne de caractères' n'est pas sensible à la casse (*indifférence aux majuscules-minuscules*). Ceci est dû à l'interclassement. Nous avons gardé l'interclassement par défaut du jeu de caractères UTF-8, qui n'est pas sensible à la casse.

Si vous désirez faire une recherche sensible à la casse, vous pouvez définir votre chaîne de recherche comme une chaîne de type binaire avec le mot clé 'BINARY' :

```
SELECT *
FROM Voiture
WHERE marque LIKE 'TO%'; -- insensible à la casse


SELECT *
FROM Voiture
WHERE marque LIKE BINARY 'TO%'; -- sensible à la casse
```

ii. OPERATEURS BETWEEN

Il est possible de faire une recherche sur un intervalle à l'aide uniquement des opérateurs de comparaison >= et <=. Par exemple, on peut rechercher les voitures qui sont immatriculées entre le 20 novembre 2021 et le 18 février 2022 de la manière suivante :

```
SELECT *
FROM voiture
WHERE datecirc >= '2021-11-20'
    AND datecirc <= '2022-02-18';
```

Cela fonctionne très bien. Cependant, SQL dispose d'un opérateur spécifique, pour les intervalles, qui pourrait vous éviter les erreurs d'inattention et rendre votre requête plus lisible et plus performante : BETWEEN.

L'utilisation est : BETWEEN *minimum* AND *maximum*.

La requête précédente peut donc s'écrire comme ceci :

```
SELECT *
FROM voiture
WHERE datecirc BETWEEN '2021-11-20'  AND '2022-02-18';
```

BETWEEN peut s'utiliser avec des dates, mais aussi avec des nombres (*BETWEEN 0 AND 100*) ou avec des chaînes de caractères (BETWEEN 'a' AND 'd'), auquel cas c'est l'ordre alphabétique qui sera utilisé (toujours insensible à la casse sauf si l'on utilise des chaînes binaires : BETWEEN BINARY 'a' AND BINARY 'd').

Bien évidemment, on peut aussi exclure un intervalle avec NOT BETWEEN.

iii. OPERATEURS IN

Cet opérateur vous permet de faire des recherches dans une liste de valeurs. Parfait par exemple pour rechercher les voitures correspondant à une liste de marques.

Imaginons que vous vouliez récupérer les informations sur les voitures dont les marques sont les suivantes : TOYOYA, KIA, HYUNDAI, SUZUKI, RENAULT. Jusqu'à maintenant, vous auriez sans doute fait quelque chose comme ça :

```
SELECT *
FROM Voiture
WHERE marque = 'TOYOTA'
    OR marque = 'KIA'
    OR marque = 'HYUNDAI'
    OR marque = 'SUZUKI'
    OR marque = 'RENAULT';
```

C'est un peu fastidieux, utilisez plutôt IN

Voici la manière d'utiliser IN :

```
SELECT *
FROM Voiture
WHERE marque IN ('TOYOTA', 'KIA', 'HYUNDAI', 'SUZUKI', 'RENAULT');
```

d. SUPPRIMER ET MODIFIER DES DONNEES

Après avoir insérer des données, sélectionner des données et ordonner des données selon les critères au choix, on peut maintenant aborder la **Suppression** et la **Modification** des données.

Par principe, il est fortement recommandé de faire une sauvegarde régulière des données. Avant de commencer les opérations de modification et de suppression, penser à une sauvegarde des données. C'est IMPORTANT !

Sans sauvegarde, toutes vos données peuvent disparaitre pour jamais à la suite d'une mauvaise manipulation.

La sauvegarde et la sauvegarde des données seront étudiées dans le module « bases de données avancées »

i. SUPPRESSION DES DONNEES

La commande utilisée pour supprimer des données est DELETE. Cette opération est irréversible, soyez très PRUDENT !

On utilise la clause WHERE de la même manière qu'avec la commande SELECT pour préciser quelles lignes doivent être supprimées.

```
DELETE FROM nom_table
WHERE critères;
```

Par exemple : La voiture immatriculée TG-5220-BA a été revendu. Nous allons donc la retirer de la base de données.

```
DELETE FROM Voiture
WHERE immat='TG-5220-AB';
```

Si vous désirez supprimer toutes les lignes d'une table, il suffit de ne pas préciser de clause WHERE.

```
DELETE FROM Voiture;
```

Attention, cette opération est irréversible. Soyez toujours bien sûr d'avoir sous la main une sauvegarde de votre base de données au cas où vous regretteriez votre geste si ce n'est pas ce que vous vouliez faire.

ii. MODIFICATION DES DONNEES

La modification des données se fait grâce à la commande UPDATE, dont la syntaxe est la suivante :

```
UPDATE nom_table
SET col1 = val1 [, col2 = val2, ...]
[WHERE ...];
```

Par exemple, vous vous êtes rendu compte que le prix d'achat de la voiture immatriculée TG-0001-BD est erroné dans la base de données. Il vous faut donc modifier le prix d'achat Voici la requête qui va vous le permettre :

```
UPDATE Voiture
SET pxachat=13000000
WHERE immat ='TG-0001-BD';
```

Tout comme pour la commande DELETE, si vous omettez la clause WHERE dans un UPDATE, la modification se fera sur toutes les lignes de la table.

La requête suivante changerait donc la date de mise en circulation de toutes les voitures stockées dans la base (ne l'exécutez pas).

```
UPDATE Voiture
SET datecirc='2022-03-21 08:00:00';
```

e. CLES PRIMAIRES ET ETRANGERES

Les CLES sont intimement liées aux index. Et tout comme NOT NULL et les index UNIQUE, les clés font partie de ce que l'on appelle les CONTRAINTES.

Il existe deux types de clés :

- CLES PRIMAIRES : notions déjà été survolées lors du chapitre sur la création d'une table, et qui servent à identifier une ligne de manière unique dans une table ;
- CLES ETRANGERES : permettent de gérer des relations entre plusieurs tables, et garantissent la cohérence des données.

i. CLES PRIMAIRES

La clé primaire d'une table est une contrainte d'unicité, composée d'une ou plusieurs colonnes, et qui permet d'identifier de manière unique chaque ligne de la table.

- CONTRAINTE D'UNICITE : ceci ressemble à un index UNIQUE ;
- COMPOSITES : Composée d'une ou plusieurs colonnes comme les index ;

- Permet d'identifier chaque ligne de manière unique : dans ce cas, une clé primaire ne peut pas être NULL.

Une clé primaire est un index UNIQUE sur une colonne qui ne peut pas être NULL.

➢ ***Création d'une clé primaire***

- ***Lors de la création de la table***

On peut donc préciser PRIMARY KEY dans la description de la colonne qui doit devenir la clé primaire (*pas de clé composite, dans ce cas*) :

```
CREATE TABLE [IF NOT EXISTS] Nom_table (
colonne1 description_colonne1 PRIMARY KEY [,
colonne2 description_colonne2,
colonne3 description_colonne3,
    ...,]
);
```

Exemple :

```
CREATE TABLE voiture (
       id SMALLINT AUTO_INCREMENT PRIMARY KEY,
       immat VARCHAR(10) NULL,
       marque VARCHAR(15) NOT NULL,
       moteur VARCHAR(15) NOT NULL,
       datecir DATETIME NULL,
       km INT NOT NULL,
       prix INT NOT NULL,
       nbreplace MALLINT NOT NULL
);
```

Ou bien, on ajoute la clé à la suite des colonnes. Cette méthode permet bien sûr la création d'une clé composite (*avec plusieurs colonnes*).

```
       CREATE TABLE [IF NOT EXISTS]
       Nom_table (     colonne1
       description_colonne1 [,
       colonne2 description_colonne2,
       colonne3 description_colonne3,
           ...],
       [CONSTRAINT [symbole_contrainte]] PRIMARY KEY (colonne_pk1 [, colonne_pk2, ...])     --
comme pour les index UNIQUE, CONSTRAINT est facultatif );
```

Exemple :

```
CREATE TABLE voiture (
       id SMALLINT AUTO_INCREMENT,
       immat VARCHAR(10) NULL,
       marque VARCHAR(15) NOT NULL,
       moteur VARCHAR(15) NOT NULL,
       datecir DATETIME NULL,
       km INT NOT NULL,
       prix INT NOT NULL,
       nbreplace MALLINT NOT NULL,
       PRIMARY KEY (id)
);
```

- ***Après création de la table***

On peut toujours utiliser ALTER TABLE. Si vous créez une clé primaire sur une table existante, assurez-vous que la ou les colonnes où vous souhaitez l'ajouter ne contiennent pas NULL.

```
ALTER TABLE nom_table
ADD [CONSTRAINT [symbole_contrainte]] PRIMARY KEY (colonne_pk1 [, colonne_pk2, ...]);
```

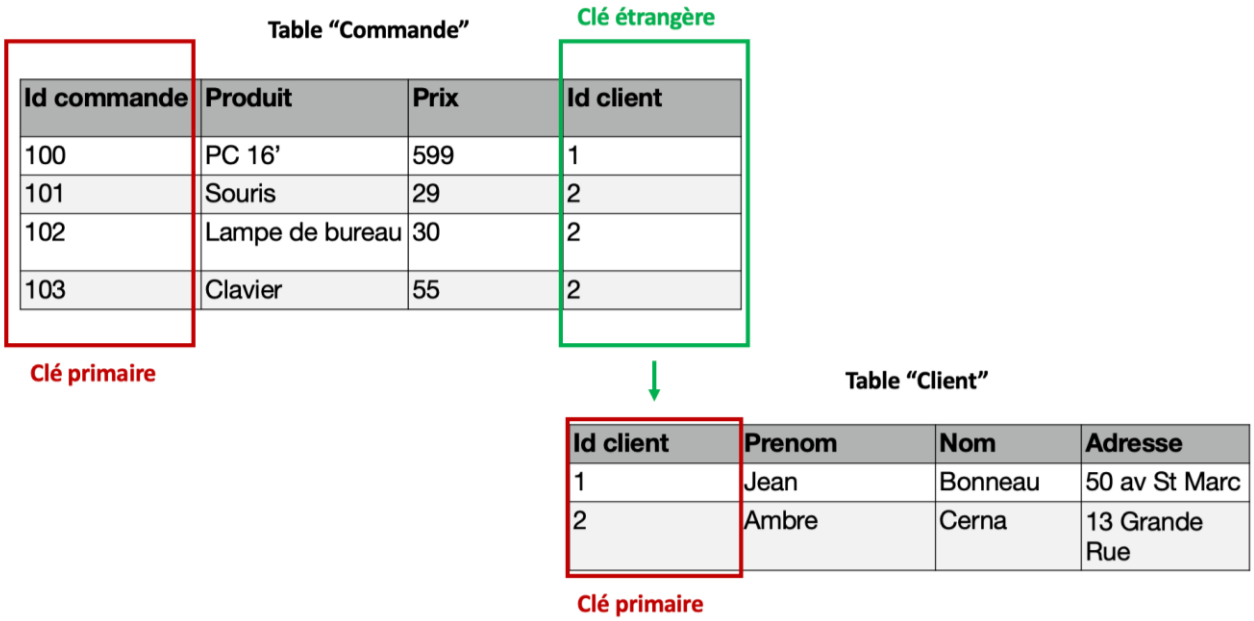
➢ ***Suppression d'une clé primaire***

```
ALTER TABLE nom_table
DROP PRIMARY KEY
```

ii. CLES ETRANGERES

Les clés étrangères ont pour fonction principale la vérification de l'intégrité de votre base. Elles permettent de s'assurer que vous insérez de bonnes données.

Exemple :



Id commande	Produit	Prix	Id client
100	PC 16'	599	1
101	Souris	29	2
102	Lampe de bureau	30	2
103	Clavier	55	2

Id client	Prenom	Nom	Adresse
1	Jean	Bonneau	50 av St Marc
2	Ambre	Cerna	13 Grande Rue

Il s'agit dans l'exemple de vérifier à l'insertion d'une ligne dans la table Commande, que le numéro de client indiqué correspond bien à un client de la table Client.

Ce numéro client qui est la clé de la table Client passe dans la table Commande au moment d'une commande : c'est donc une "Clé étrangère".

Par conséquent, si une Clé étrangère est créée sur la colonne client de la table Commande en lui donnant comme référence la colonne Numéro de la table Client, vous ne pourrez plus jamais insérer un numéro de client inexistant dans la table Commande. Il s'agit bien sûr d'une contrainte !

QUELQUES POINTS IMPORTANTS :

- Comme pour les index et les clés primaires, il est possible de créer des Clés étrangères composites ;
- Lorsqu'une Clé étrangère est créée sur une colonne (*ou un groupe de colonnes*), un index est automatiquement ajouté sur celle-ci (*ou sur le groupe*) ;
- Cependant, la colonne (le groupe de colonnes) qui sert de référence doit déjà posséder un index (ou être clé primaire) ;

- La colonne (*ou le groupe de colonnes*) sur laquelle (lequel) la Clé est créée doit être exactement du même type que la colonne (*le groupe de colonnes*) qu'elle (il) référence.
- Sous MySQL, le moteur de table MyISAM ne permet pas l'utilisation des clés étrangères, contrairement à InnoDB.

➢ *Création d'une clé étrangère*

 La ou les colonnes sur lesquelles on crée la clé - on utilise FOREIGN KEY; La ou les colonnes qui vont servir de référence - on utilise REFERENCES.

○ *Lors de la création de la table*

```
CREATE TABLE [IF NOT EXISTS] Nom_table (
colonne1 description_colonne1,
[colonne2 description_colonne2,
colonne3 description_colonne3,
    ...,]
    [ [CONSTRAINT [symbole_contrainte]]  FOREIGN KEY (colonne(s)_clé_étrangère)
                                    REFERENCES table_référence (colonne(s)_référence)]
);
```

Exemple : Si l'on imagine les tables *Client* et *Commande*, pour créer la table *Commande* avec une clé étrangère ayant pour référence la colonne *numero* de la table *Client*, on utilisera :

```
CREATE TABLE Commande (
Id_Commande INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
produit VARCHAR(40),
prix INT UNSIGNED NOT NULL,
Id_client INT UNSIGNED NOT NULL,
    CONSTRAINT fk_client_numero          -- On donne un nom à notre clé
        FOREIGN KEY (Id_client)             -- Colonne sur laquelle on crée la clé
        REFERENCES Client(Id_client)        -- Colonne de référence )
ENGINE=InnoDB;                           -- MyISAM interdit, je le rappelle encore une fois
```

○ *Après création de la table*

Tout comme pour les clés primaires, pour créer une clé étrangère après création de la table, il faut utiliser ALTER TABLE.

```
ALTER TABLE Commande
ADD CONSTRAINT fk_client_numero FOREIGN KEY (client) REFERENCES Client(numero);
```

➢ *Suppression d'une clé étrangère*

Il peut y avoir plusieurs clés étrangères par table. Par conséquent, lors d'une suppression, il faut identifier la clé à détruire. Cela se fait grâce au symbole de la contrainte.

```
ALTER TABLE nom_table
DROP FOREIGN KEY symbole_contrainte
```

Pratiques : Modification de notre base PARC_AUTO

- **Ajoutons la table Marque**

```
CREATE TABLE Marque (
codemarque VARCHAR(3) NOT NULL,
nommarque VARCHAR(15) NOT NULL,
paysmarque VARCHAR(15) NOT NULL,
PRIMARY KEY(codemarque)
);

INSERT INTO marque (codemarque, nommarque, paysmarque)
VALUES
INSERT INTO marque (codemarque, nommarque, paysmarque)
VALUES
   ('TOY', 'TOYOTA', 'JAPON'),
   ('MIT', 'MITSUBISHI', 'JAPON'),
   ('MAZ', 'MAZDA', 'JAPON'),
   ('NIS', 'NISSAN', 'JAPON'),
   ('SUZ', 'SUZUKI', 'JAPON'),
   ('BEN', 'MERCEDES BENZ', 'ALLEMAGNE'),
   ('FOR', 'FORD', 'ETATS-UNIS'),
   ('BMW', 'BMW', 'ALLEMAGNE'),
   ('KIA', 'KIA', 'COREE DU SUD'),
   ('HYU', 'HYUNDAI', 'COREE DU SUD'),
   ('REN', 'RENAULT', 'FRANCE'),
   ('PEU', 'PEUGEOT', 'FRANCE'),
   ('TES', 'TESLA', 'ETATS-UNIS'),
   ('VOL', 'VOLKSWAGEN', 'ALLEMAGNE'),
   ('FIA', 'FIAT', 'ITALIE'),
   ('LEX', 'LEXUS', 'JAPON'),
   ('JEE', 'JEEP', 'ETATS-UNIS'),
   ('ROL', 'ROLLS ROYCE', 'ROYAUME-UNI'),
   ('MG', 'MG', 'ROYAUME-UNI'),
   ('LAM', 'LAMBORGHINI', 'ITALIE'),
   ('CIT', 'CITROEN', 'FRANCE'),
   ('AUD', 'AUDI', 'ALLEMAGNE'),
   ('FER', 'FERRARI', 'ITALIE'),
   ('SUB', 'SUBARU', 'JAPON');
```

- **Modification de la table Voiture pour intégrer codemarque de la table Marque.**

```
-- Ajout de la colonne codemarque
ALTER TABLE voiture
ADD COLUMN codemarque VARCHAR (3); -- même type que la colonne codemarque de marque
-- Remplissage de codemarque
UPDATE Voiture SET codemarque = 'BEN' WHERE marque = 'MERCEDES BENZ';
UPDATE Voiture SET codemarque = 'TOY' WHERE marque = 'TOYOTA';
UPDATE Voiture SET codemarque = 'SUZ' WHERE marque = 'SUZUKI';
UPDATE Voiture SET codemarque = 'HYU' WHERE marque = 'HYUNDAI';
UPDATE Voiture SET codemarque = 'BMW' WHERE marque = 'BMW';
UPDATE Voiture SET codemarque = 'MIT' WHERE marque = 'MITSUBISHI';
UPDATE Voiture SET codemarque = 'MAZ' WHERE marque = 'MAZDA';
UPDATE Voiture SET codemarque = 'KIA' WHERE marque = 'KIA';
UPDATE Voiture SET codemarque = 'NIS' WHERE marque = 'NISSAN';
UPDATE Voiture SET codemarque = 'REN' WHERE marque = 'RENAULT';
UPDATE Voiture SET codemarque = 'FOR' WHERE marque = 'FORD';
UPDATE Voiture SET codemarque = 'PEU' WHERE marque = 'PEUGEOT';
UPDATE Voiture SET codemarque = 'TES' WHERE marque = 'TESLA';
UPDATE Voiture SET codemarque = 'VOL' WHERE marque = 'VOLKSWAGEN';
UPDATE Voiture SET codemarque = 'FIA' WHERE marque = 'FIAT';
UPDATE Voiture SET codemarque = 'LEX' WHERE marque = 'LEXUS';
UPDATE Voiture SET codemarque = 'JEE' WHERE marque = 'JEEP';
UPDATE Voiture SET codemarque = 'ROL' WHERE marque = 'ROLLS ROYCE';
UPDATE Voiture SET codemarque = 'MG' WHERE marque = 'MG';
UPDATE Voiture SET codemarque = 'LAM' WHERE marque = 'LAMBORGHINI';
UPDATE Voiture SET codemarque = 'CIT' WHERE marque = 'CITROEN';
UPDATE Voiture SET codemarque = 'AUD' WHERE marque = 'AUDI';
UPDATE Voiture SET codemarque = 'FER' WHERE marque = 'FERRARI';
UPDATE Voiture SET codemarque = 'SUB' WHERE marque = 'SUBARU';

-- Suppression de la colonne marque
ALTER TABLE voiture
DROP COLUMN marque;

-- Ajout de la clé étrangère
ALTER TABLE Voiture
ADD CONSTRAINT fk_codemarque FOREIGN KEY (codemarque) REFERENCES Marque(codemarque);
```

;

TESTS : Essayons d'ajouter une voiture dont le code de la marque est BUG (qui n'existe donc pas dans la table marque) :

```
INSERT INTO Voiture (immat, moteur, datecirc, km, pxachat, nbreplace, codemarque)
VALUES ('TG-4444-BK','HYBRIDE','2022-12-12 08:00:00', 0,55000000,5,'BUG');
```

RESULTATS

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint
fails (`parc_auto`.`voiture`, CONSTRAINT `fk_codemarque` FOREIGN KEY
(`codemarque`) REFERENCES `marque` (`codemarque`))
mysql>
```

➢ *OPTIONS DES CLES ETRANGERES*

Les clés étrangères ont deux options fort utiles :

- ON DELETE : qui permet de déterminer le comportement de MySQL en cas de suppression d'une référence ;
- ON UPDATE : qui permet de déterminer le comportement de MySQL en cas de modification d'une référence.

Que se passe-t-il si l'on supprime la marque KIA ? Certaines voitures référencent cette marque dans leur colonne codemarque. On risque donc d'avoir des données incohérentes. Or, éviter cela est précisément la raison d'être de notre clé étrangère.

Essayons :

```
DELETE FROM Marque WHERE codemarque = 'KIA';

mysql> DELETE FROM Marque WHERE codemarque = 'KIA';
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint
fails (`parc_auto`.`voiture`, CONSTRAINT `fk_codemarque` FOREIGN KEY
(`codemarque`) REFERENCES `marque` (`codemarque`))
mysql>
```

MySQL vérifie la contrainte de clé étrangère lors d'une suppression ou d'une modification et empêche de supprimer ou de modifier une ligne si elle contient une référence utilisée ailleurs dans la base.

C'est dans ces cas qu'interviennent les options ON DELETE et ON UPDATE, qui permettent de changer la manière dont la clé étrangère gère la suppression et la modification d'une référence.

SYNTAXE

```
ALTER TABLE nom_table
ADD [CONSTRAINT fk_col_ref]
    FOREIGN KEY (colonne)
    REFERENCES table_ref(col_ref)
    ON DELETE {RESTRICT | NO ACTION | SET NULL | CASCADE}
    OU
    ON UPDATE {RESTRICT | NO ACTION | SET NULL | CASCADE};
```

Il y a donc quatre comportements possibles : RESTRICT, NO ACTION, SET NULL et CASCADE.

- RESTRICT ou NO ACTION : RESTRICT est le comportement par défaut. Si l'on essaye de supprimer ou de modifier une valeur référencée par une clé étrangère, l'action est avortée et on obtient une erreur. NO ACTION a exactement le même effet.

  Cette équivalence de RESTRICT et NO ACTION est propre à MySQL. Dans d'autres SGBD, ces deux options n'auront pas le même effet (RESTRICT étant généralement plus strict que NO ACTION).

- SET NULL : Si l'on choisit SET NULL, alors tout simplement, NULL est substitué aux valeurs dont la référence est supprimée.

- CASCADE : supprime ou modifie toutes les lignes qui référençaient la valeur supprimée.

4. JOINTURES

Les jointures permettent de manipuler plusieurs tables dans la même requête.
Elles peuvent être faites avec ou sans JOIN.

a. JOINTURES SANS JOIN

Il est possible de faire une jointure sans utiliser le mot JOIN. Ce n'est cependant possible que pour les jointures internes.
Il suffit de mentionner les tables que l'on veut joindre dans la clause FROM (séparées par des virgules), et de mettre la condition de jointure dans la clause WHERE (sans clause ON).

```
SELECT *
FROM table1, table2
WHERE table1.colonne1 = table2.colonne1;

-- équivalent à

SELECT *
FROM table1
[INNER] JOIN table2
    ON table1.colonne1 = table2.colonne1 ;
```

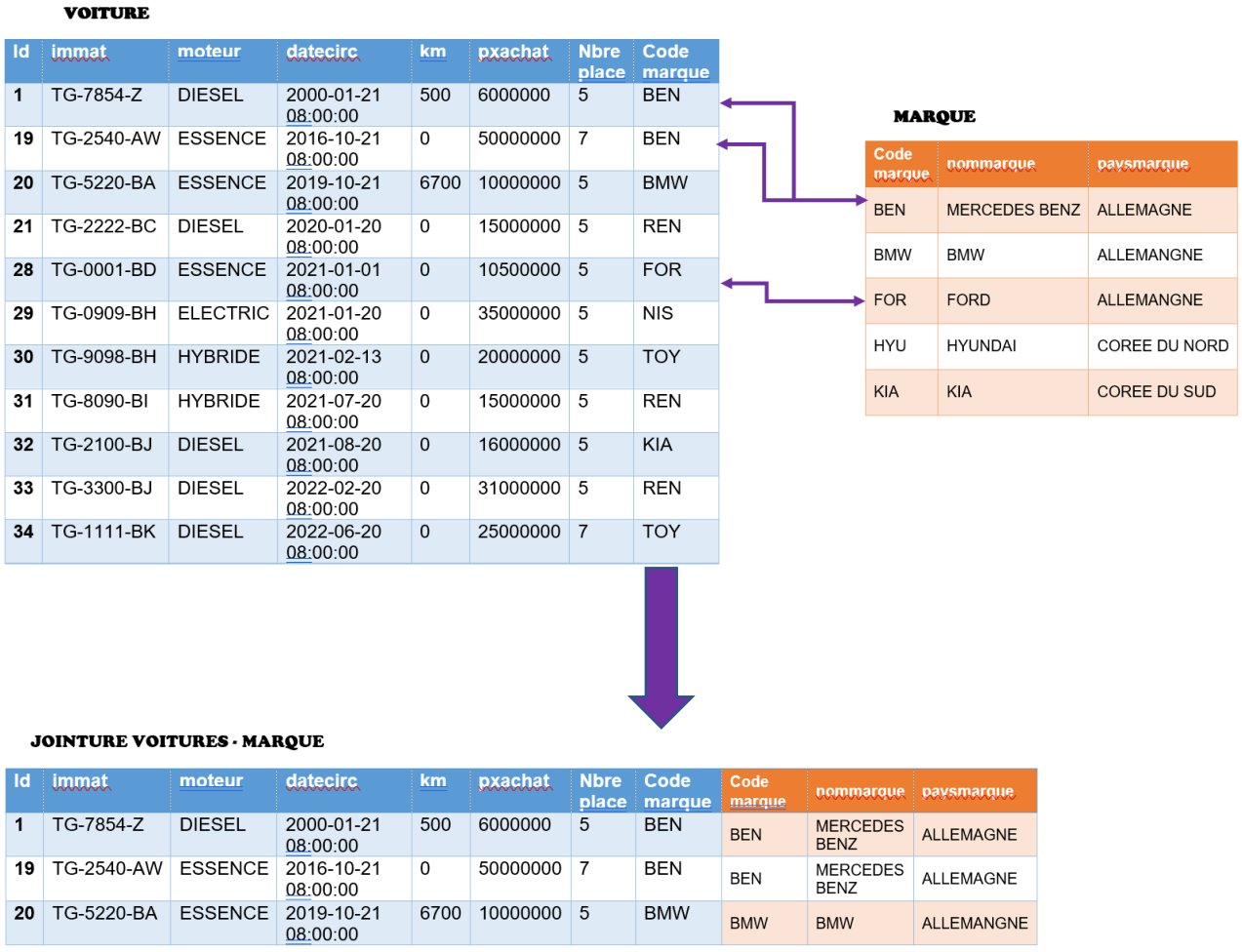
b. PRINCIPE DES JOINTURES

Il s'agit de joindre plusieurs tables. Pour ce faire, on utilise les informations communes de ces tables.
En fait, lorsque l'on fait une jointure, on crée une table virtuelle et temporaire qui reprend les colonnes des tables liées.

Pour le cas des tables *Voitures* et Marques vues précédemment, une fois les deux tables jointes, on obtient une table possédant toutes les colonnes de voiture et toutes les colonnes de Marque, avec les valeurs correspondantes de chaque table.

De cette table virtuelle, on peut extraire ce que l'on veut. La colonne paysmarque pour la ligne ayant "TG-0001-BD" dans la colonne *immat*, par exemple.

Le schéma ci-dessous illustre ce principe.



c. *NOTION D'ALIAS*

Les alias sont des noms de remplacement que l'on donne de manière temporaire (le temps d'une requête) à une colonne, une table, une donnée. Les alias sont introduits par le mot-clé AS.

Exemple :

```
SELECT immat AS IMMATRICULATION, km AS KILOMETRAGE, datecirc AS 'DATE MISE EN
CIRCULATION'
FROM Voiture
WHERE IMMAT is not null;
```

IMMATRICULATION	KILOMETRAGE	DATE MISE EN CIRCULATION
TG-7854-Z	500	2000-01-21 08:00:00
TG-6733-AB	0	2002-01-21 18:00:00
TG-2020-AC	0	2003-03-21 08:00:00
TG-3733-AC	0	2010-01-21 18:00:00
TG-6733-AC	0	2012-01-21 18:00:00
TG-5420-AF	10000	2013-01-21 18:00:00
TG-9420-AF	0	2013-09-21 18:00:00
TG-1200-AK	0	2014-01-21 18:00:00
TG-9420-AK	100	2014-09-21 18:00:00
TG-0020-AL	0	2014-11-21 18:00:00
TG-4444-AL	5	2015-09-21 18:00:00
TG-5420-AL	0	2015-10-21 08:00:00
TG-0020-AV	30	2018-10-21 08:00:00
TG-0480-AV	0	2015-12-21 08:00:00
TG-1420-AW	0	2016-02-01 08:00:00
TG-2540-AW	0	2016-10-21 08:00:00
TG-5220-BA	6700	2019-10-21 08:00:00
TG-2222-BC	0	2020-01-20 08:00:00
TG-0001-BD	0	2021-01-01 08:00:00
TG-0909-BH	0	2021-01-20 08:00:00
TG-9098-BH	0	2021-02-13 08:00:00
TG-8090-BI	0	2021-07-20 08:00:00
TG-2100-BJ	0	2021-08-20 08:00:00
TG-3300-BJ	0	2022-02-20 08:00:00
TG-1111-BK	0	2022-06-20 08:00:00

25 rows in set (0.00 sec)

d. JOINTURES INTERNE : INNER JOIN

Une jointure interne permet de sélectionner les enregistrements ayant des correspondances entre deux fichiers joints. Lorsque l'on fait une jointure interne, on exige qu'il y ait des données de part et d'autre de la jointure.

Par exemple, pour lister les clients ayant déjà passé des commandes, il est nécessaire de lier les fichiers "Client" et "Commande" grâce à une jointure interne. La requête sélectionnera uniquement les clients liés à au moins un numéro de commande. Les clients n'ayant jamais passé de commande ne seront pas listés.

Exemple : Sélectionner les marques auxquelles correspond au moins une voiture.

Pour des vérifications insérer deux autres marques dans la table Marque.

```
INSERT INTO Marque (codemarque, nommarque, paysmarque)
VALUES
   ('VLV','VOLVO','SUEDE'),
   ('BUG','BUGATTI','FRANCE');
```

La table Marque se présentera alors ainsi :

codemarque	nommarque	paysmarque
AUD	AUDI	ALLEMAGNE
BEN	MERCEDES BENZ	ALLEMAGNE
BMW	BMW	ALLEMAGNE
BUG	BUGATTI	FRANCE
CIT	CITROEN	FRANCE
FER	FERRARI	ITALIE
FIA	FIAT	ITALIE
FOR	FORD	ETATS-UNIS
HYU	HYUNDAI	COREE DU SUD
JEE	JEEP	ETATS-UNIS
KIA	KIA	COREE DU SUD
LAM	LAMBORGHINI	ITALIE
LEX	LEXUS	JAPON
MAZ	MAZDA	JAPON
MG	MG	ROYAUME-UNI
MIT	MITSUBISHI	JAPON
NIS	NISSAN	JAPON
PEU	PEUGEOT	FRANCE
REN	RENAULT	FRANCE
ROL	ROLLS ROYCE	ROYAUME-UNI
SUB	SUBARU	JAPON
SUZ	SUZUKI	JAPON
TES	TESLA	ETATS-UNIS
TOY	TOYOTA	JAPON
VLV	VOLVO	SUEDE
VOL	VOLKSWAGEN	ALLEMAGNE

26 rows in set (0.00 sec)

Exécuter ensuite la requête

```
SELECT distinct marque.codemarque, nommarque
FROM   Marque
INNER JOIN Voiture
      ON marque.codemarque = voiture.codemarque;
```

RESULTAT :

codemarque	nommarque
AUD	AUDI
BMW	BMW
CIT	CITROEN
FER	FERRARI
FIA	FIAT
FOR	FORD
HYU	HYUNDAI
JEE	JEEP
KIA	KIA
LAM	LAMBORGHINI
LEX	LEXUS
MG	MG
MIT	MITSUBISHI
NIS	NISSAN
PEU	PEUGEOT
ROL	ROLLS ROYCE
SUB	SUBARU
SUZ	SUZUKI
TES	TESLA
TOY	TOYOTA
VOL	VOLKSWAGEN

21 rows in set (0.00 sec)

Effectivement le résultat donne la liste des Marques auxquelles correspond au moins une voiture. Nous avons enregistré 14 marques au total, mais la requête précédente ne retourne que 12. VOLVO et BUGATTI n'apparaissant pas dans la liste.

Décomposons la requête

- SELECT distinct marque.codemarque, nommarque : pour sélectionner les colonnes *marque et nommarque* de la table Marque (*distinct pour éliminer les doublons) ;*
- FROM Marque : sur la table Marque ;
- INNER JOIN Voiture : joindre la table Marque (*avec une jointure interne*) à la table Voiture *;*
- ON Marque.codemarque= Voiture.codemarque : jointure faite sur les colonnes *codemarque* de la table Marque et codemarque de la table VOiture, qui doivent donc correspondre.

SYNTAXE

```
SELECT *                                    -- comme d'habitude, vous sélectionnez
les colonnes que vous voulez
FROM nom_table1
[INNER] JOIN nom_table2                     -- INNER explicite le fait qu'il s'agit
                                               d'une jointure interne, mais c'est
                                               facultatif
        ON colonne_table1 = colonne_table2  -- sur quelles colonnes se fait la
                                               jointure
                                            -- vous pouvez mettre colonne_table2 =
                                               colonne_table1, l'ordre n'a pas
                                               d'importance
[INNER] JOIN nom_tableX
[ON ...]
[WHERE ...]
[ORDER BY ...]                              -- les clauses habituelles sont bien sûr
                                               utilisables !
[LIMIT ...]
```

e. *UTILISATION DES ALIAS*

Les alias sont souvent utilisés avec les jointures. Ils permettent notamment de renommer les tables, et ainsi d'écrire moins de code.

Exemple : on renomme la table Marque "*M*", et la table Voiture "*V*".

```
SELECT M.codemarque, M.nommarque, V.immat
FROM   Marque M
INNER JOIN Voiture V
      ON M.codemarque = V.codemarque
WHERE V.immat LIKE '%AK';
```

Nous allons modifier la base de données pour pouvoir exécuter les commandes à suivre.
Nous ajouterons une table « PAYS » pour gérer les pays de provenances des différentes marques.
Avant ça supprimons le chams pays marque de la table Marque.

f. FONCTIONS D'AGREGATION

Les fonctions d'agrégation, ou de groupement, sont des fonctions qui vont regrouper les lignes. Elles agissent sur une colonne et renvoient un résultat unique pour toutes les lignes sélectionnées (ou pour chaque groupe de lignes, mais nous verrons cela plus tard).

i. COUNT

La fonction COUNT() permet de savoir combien de lignes sont sélectionnées par la requête.

```
--combien de marques de voitures disposons nous ?
SELECT COUNT(*)  AS 'Nombre Marques'
FROM   Marque ;
--combien de voitures de marque BUGATTI disposons nous ?
SELECT COUNT(*) AS Nbre_Bugatti
FROM   Voiture V
JOIN Marque M ON V.codemarque = M.codemarque
WHERE M.nommarque = 'BUGATTI' ;
```

COUNT(*) permet de compter toutes les lignes, sans se soucier de ce qu'elles contiennent.
COUNT(colonne), permet de compter les lignes dont la valeur de colonne n'est pas NULL.

Comptons le nombre de voitures et le nombre de voitures immatriculées de la table Voiture.

```
SELECT COUNT(*) AS Nbre_voiture, COUNT(immat) AS Nbre_voiture_immatric
FROM   Voiture ;
```

Nbre_voiture	Nbre_voiture_immatric
60	47

Nous avons donc 25 voitures immatriculées sur les 34 du parc.

ii. DISTINCT

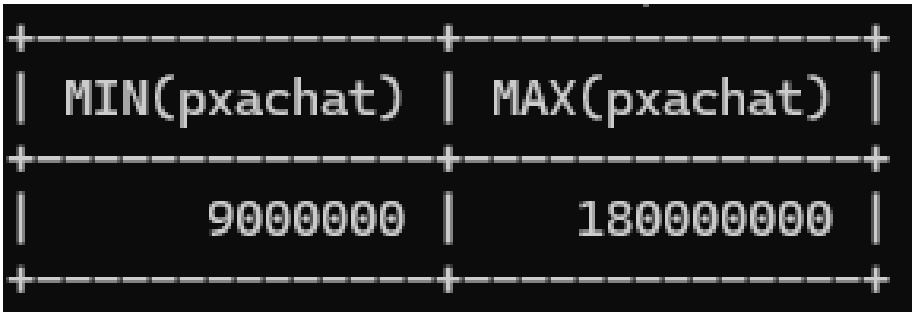
Il est utilisé pour éliminer les doublons.
Exemple : comptons le nombre de marques distinctes définies dans la table Voitures.

```
SELECT COUNT(DISTINCT codemarque)
FROM   Voiture ;
```

iii. MIN et MAX

La fonction MIN(x) retourne la plus petite valeur de x et MAX(x) la plus grande valeur de x.

```
SELECT MIN(pxachat), MAX(pxachat)
FROM   Voiture ;
```



MIN(pxachat)	MAX(pxachat)
9000000	180000000

Notez que MIN() et MAX() ne s'utilisent pas uniquement sur des données numériques. Si vous lui passez des chaînes de caractères, MIN() récupérera la première chaîne dans l'ordre alphabétique, MAX() la dernière. Avec des dates, MIN() renverra la plus ancienne et MAX() la plus récente.

```
SELECT MIN(immat), MAX(immat), MIN(datecirc), MAX(datecirc)
FROM Voiture;
```

iv. SUM

La fonction SUM(x) renvoie la somme de x.

```
SELECT SUM(pxachat)
FROM Voiture;
```

v. AVG

La fonction AVG(x) (du mot anglais average) renvoie la valeur moyenne de x.

```
SELECT AVG(pxachat)
FROM Voiture;
```

5. REGROUPEMENT

a. LA CLAUSE GROUP BY

Elle permet de réaménager le tableau résultant du SELECT en un nombre minimum de groupe tel qu'à l'intérieur de chaque groupe l'attribut spécifié a la même valeur sur chaque ligne. Lorsqu'une fonction de calcul précède cette clause, elle est calculée pour chaque groupe.

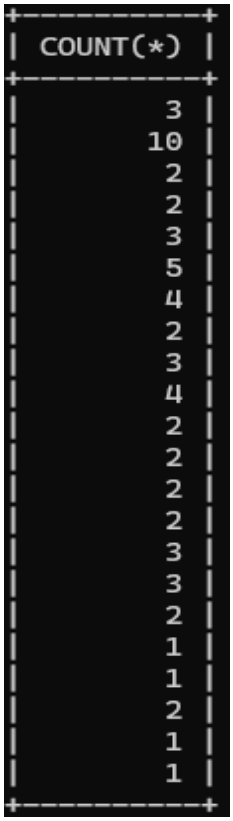
Imaginons que nous souhaiterions avoir disposer des groupes par marques et savoir en une seule requête combien de TOYOTA, BMW, SUZIKI, etc. nous disposons ? Un simple groupement, et c'est fait !

Pour regrouper les lignes selon un critère, il faut utiliser GROUP BY, qui se place après l'éventuelle clause WHERE (sinon directement après FROM), suivi du nom de la colonne à utiliser comme critère de regroupement.

```
SELECT ...
FROM nom_table
[WHERE condition]
GROUP BY nom_colonne;
```

Exemple 1 : Comptons les lignes dans la table Voiture, en regroupant sur le critère de la marque (donc avec la colonne codemarque).

```
SELECT COUNT(*)
FROM Voiture
GROUP BY codemarque;
```



COUNT(*)
3
10
2
2
3
5
4
2
3
4
2
2
2
2
3
3
2
1
1
2
1
1

Exécution la même requête en considérant juste les voitures diésel.

```
SELECT COUNT(*)
FROM Voiture
WHERE moteur LIKE 'DIESEL'
GROUP BY codemarque;
```

b. Ajout d'autres colonnes

Pour savoir à quoi correspond chaque nombre, il suffit d'afficher également le critère qui a permis de regrouper les lignes. Dans notre cas, codemarque.

```
SELECT codemarque, COUNT(*) as Nombre_voiture
FROM Voiture
GROUP BY codemarque;
```

codemarque	COUNT(*)
CIT	3
TOY	10
HYU	2
MG	2
NIS	3
AUD	5
KIA	4
LAM	2
NULL	3
BMW	4
FER	2
SUZ	2
MIT	2
PEU	2
TES	3
FOR	3
VOL	2
FIA	1
SUB	1
LEX	2
JEE	1
ROL	1

C'est déjà mieux, mais l'idéal serait d'avoir le nom des marques directement. Qu'à cela ne tienne, il suffit de faire une jointure ! Sans oublier de changer le critère et de mettre *nommarque* à la place de codemarque.

```
SELECT nommarque, COUNT(*) as Nombre_voiture
FROM   Voiture V
      JOIN Marque M
      ON M.codemarque = V.codemarque
GROUP BY nommarque;
```

nommarque	Nombre_voiture
CITROEN	3
TOYOTA	10
HYUNDAI	2
MG	2
NISSAN	3
AUDI	5
KIA	4
LAMBORGHINI	2
BMW	4
FERRARI	2
SUZUKI	2
MITSUBISHI	2
PEUGEOT	2
TESLA	3
FORD	3
VOLKSWAGEN	2
FIAT	1
SUBARU	1
LEXUS	2
JEEP	1
ROLLS ROYCE	1

c. *LA CLAUSE HAVING*

C'est l'équivalent de WHERE appliqué au GROUP BY. Elle ne peut être spécifié que si une clause GROUP BY l'a été. En effet le critère spécifié dans la clause HAVING porte sur la valeur d'une fonction calculée ou expression sur un groupe.

Il n'est pas possible d'utiliser la clause WHERE pour faire des conditions sur une fonction d'agrégation. Donc, si l'on veut afficher les marques dont on dispose de plus de 5, la requête suivante ne fonctionnera pas.

```
SELECT nommarque, COUNT(*)
FROM   Marque M
JOIN Voiture V ON V.codemarque = M.codemarque
WHERE COUNT(*)>5
GROUP BY nommarque;
```

```
ERROR 1111 (HY000): Invalid use of group function
```

Il faut utiliser une clause spéciale : HAVING. Cette clause se place juste après le GROUP BY.

```
SELECT nommarque, COUNT(*)
FROM   Marque M
JOIN Voiture V ON V.codemarque = M.codemarque
GROUP BY nommarque
HAVING COUNT(*)>4;
```

nommarque	COUNT(*)
TOYOTA	10
AUDI	5

Il est également possible d'utiliser un alias dans une condition HAVING :

```
SELECT nommarque, COUNT(*) as Nombre
FROM   Marque M
JOIN Voiture V ON V.codemarque = M.codemarque
GROUP BY nommarque
HAVING Nombre>4;
```

nommarque	Nombre
TOYOTA	10
AUDI	5

6. LES UTILISATEURS ET LEURS PRIVILEGES

a. *PRIVILEGES DES UTILISATEURS*

Lorsque l'on se connecte à MySQL, on le fait avec un utilisateur. Chaque utilisateur possède une série de privilèges relatifs aux données stockées sur le serveur : le privilège de sélectionner les données, de les modifier, de créer des objets, etc.

Ces privilèges peuvent exister à plusieurs niveaux : base de données, tables, colonnes, procédures, etc.

Exemple

```
GRANT ALL PRIVILEGES ON parc_auto.* TO 'IT_B1'@'localhost';
```

Cette requête a donné tous les droits sur la base de données *parc_auto* à l'utilisateur '*IT_B1'@'localhost'*.

i. *STOCKAGE DES UTILISATEURS ET PRIVILEGES*

Toutes ces informations sont stockées dans la base de données *mysql*.

Les utilisateurs sont stockés dans la table *user*, avec leurs privilèges globaux (*c'est-à-dire valables au niveau du serveur, sur toutes les bases de données*).

La base *mysql* possède par ailleurs quatre tables permettant de stocker les privilèges des utilisateurs à différents niveaux :

- *db* : privilèges au niveau des bases de données.
- *tables_priv* : privilèges au niveau des tables.
- *columns_priv* : privilèges au niveau des colonnes.
- *proc_priv* : privilèges au niveau des routines (procédures et fonctions stockées).
-

ii. *MODIFICATIONS*

Il est tout à fait possible d'ajouter, de modifier et de supprimer des utilisateurs en utilisant des requêtes INSERT, UPDATE ou DELETE directement sur la table *mysql.user*. De même pour leurs privilèges, avec les tables *mysql.db, mysql.tables_priv, mysql.columns_priv et mysql.procs_priv.*

Cependant, en général, on utilise plutôt des commandes dédiées à cet usage. Ainsi, pas besoin de se soucier de la structure de ces tables, et le risque d'erreur est moins grand. Ce sont ces commandes que nous allons décortiquer dans la suite de ce chapitre.

b. *CREATION, MODIFICATION ET SUPPRESSION DES UTILISATEURS*

i. *SYNTAXE POUR LA CREATION ET LA SUPPRESSION*

```
-- Création
CREATE USER 'login'@'hote' [IDENTIFIED BY 'mot_de_passe'];

-- Suppression
DROP USER 'login'@'hote';
```

➢ **UTILISATEUR**

L'utilisateur est donc défini par deux éléments :

- Son login : le login est un simple identifiant. Vous pouvez le choisir comme vous voulez. Il n'est pas obligatoire de l'entourer de guillemets, sauf s'il contient des caractères spéciaux comme – ou @. C'est cependant conseillé.
- Hôte : L'hôte est l'adresse à partir de laquelle l'utilisateur va se connecter. Si l'utilisateur se connecte à partir de la machine sur laquelle le serveur MySQL se trouve, on peut utiliser 'localhost'. Sinon, on utilise en général une adresse IP ou un nom de domaine.

Exemples

```
CREATE USER 'delegueITB1'@'localhost' IDENTIFIED BY 'wincrafts@Big'.
CREATE USER 'profside'@'192.168.66.9' IDENTIFIED BY 'vivinve3';
CREATE USER 'stafdouala'@'cdp.camer.net' IDENTIFIED BY 'zamylikili';
```

Il est également possible de permettre à un utilisateur de se connecter à partir de plusieurs hôtes différents (sans devoir créer un utilisateur par hôte) : en utilisant le joker % , on peut préciser des noms d'hôtes partiels, ou permettre à l'utilisateur de se connecter à partir de n'importe quel hôte.

```
-- leader peut se connecter à partir de n'importe quel hôte dont l'adresse IP
commence par 192.168.66.
CREATE USER 'leader'@'192.168.66.%' IDENTIFIED BY 'starfff3';
-- youssoufa peut se connecter à partir de n'importe quel hôte du domaine camer.net
CREATE USER 'youssoufa'@'%.camer.net' IDENTIFIED BY 'lovedear111';
-- yeslows peut se connecter à partir de n'importe quel hôte
CREATE USER 'yeslows'@'%' IDENTIFIED BY 'meetme&me1';
```

REMARQUE :

- Comme pour le login, les guillemets ne sont pas obligatoires, sauf si un caractère spécial est utilisé (comme le joker % par exemple). Notez que si vous ne précisez pas d'hôte, c'est comme si vous autorisiez tous les hôtes. ' yeslows '@'%' est donc équivalent à ' yeslows '.
- Les guillemets se placent indépendamment autour du login et autour de l'hôte. N'entourez pas tout par des guillemets : *CREATE USER 'delegueITB1@localhost' créera un utilisateur dont le login est "delegueITB1@localhost', autorisé à se connecter à partir de n'importe quel hôte.*

- **RENOMMER L'UTILISATEUR**

Pour modifier l'identifiant d'un utilisateur (login et/ou hôte), on peut utiliser RENAME USER *ancien_utilisateur* TO *nouvel_utilisateur*.

Exemple : on renomme *delegueITB1* en seniorITB1, en gardant le même hôte.

```
RENAME USER 'delegueITB1'@'localhost' TO 'seniorITB1'@'localhost';
```

- **MOT DE PASSE**

Le mot de passe de l'utilisateur est donné par la clause IDENTIFIED BY. Cette clause n'est pas obligatoire, auquel cas l'utilisateur peut se connecter sans donner de mot de passe. Ce n'est évidemment pas une bonne idée du point de vue de la sécurité. *Évitez au maximum les utilisateurs sans mot de passe.*

Lorsqu'un mot de passe est précisé, il n'est pas stocké tel quel dans la table *mysql.user*. Il est d'abord hashé, et c'est cette valeur hashée qui est stockée.

- **MODIFIER LE MOT DE PASSE**

Pour modifier le mot de passe d'un utilisateur, on peut utiliser la commande

SET PASSWORD (à condition d'avoir les privilèges nécessaires, ou d'être connecté avec l'utilisateur dont on veut changer le mot de passe).

Cependant, cette commande ne hashe pas le mot de passe automatiquement. Il faut donc utiliser la fonction PASSWORD ().

Exemple

```
SET PASSWORD FOR 'leader'@'194.28.12.%' = PASSWORD('ititurun69');
```

c. *LES PRIVILEGES*

Lorsque l'on crée un utilisateur avec CREATE USER, celui-ci n'a au départ aucun privilège, aucun droit.

En SQL, avoir un privilège, c'est avoir l'autorisation d'effectuer une action sur un objet. Un utilisateur sans aucun privilège ne peut rien faire d'autre que se connecter. Il n'aura pas accès aux données, ne pourra créer aucun objet (base/table/procédure/autre) ni en utiliser.

i. *LES DIFFERENTS PRIVILEGES*

Il existe de nombreux privilèges dont voici une sélection des plus utilisés (à l'exception des privilèges particuliers ALL, USAGE et GRANT OPTION).

Les privilèges SELECT, INSERT, UPDATE et DELETE permettent aux utilisateurs d'utiliser ces mêmes commandes.

Privilèges concernant les tables, les vues et les bases de données

Privilège	Action autorisée
CREATE TABLE	Création de tables
CREATE TEMPORARY	Création de tables temporaires
CREATE VIEW	Création de vues (il faut également avoir le privilège SELECT sur les colonnes sélectionnées par la vue)
ALTER	Modification de tables (avec ALTER TABLE)
DROP	Suppression de tables, vues et bases de données

ii. *LES DIFFERENTS NIVEAUX D'APPLICATION DES PRIVILEGES*

Lorsque l'on accorde un privilège à un utilisateur, il faut également préciser à quoi s'applique ce privilège.

Privilège	Action autorisée
.	Privilège global : s'applique à toutes les bases de données, à tous les objets. Un privilège de ce niveau sera stocké dans la table *mysql.user*.
*	Si aucune base de données n'a été préalablement sélectionnée (avec USE nom_bdd), c'est l'équivalent de *.* (privilège stocké dans mysql.user). Sinon, le privilège s'appliquera à tous les objets de la base de données que l'on utilise (et sera stocké dans la table mysql.db).
*nom_bdd.**	Privilège de base de données : s'applique à tous les objets de la base *nom_bdd* (stocké dans *mysql.db*).
nom_bdd.nom_table	Privilège de table (stocké dans *mysql.tables_priv*).
nom_table	*Privilège de table : s'applique à la table nom_table de la base de données dans laquelle on se trouve, sélectionnée au préalable avec USE nom_bdd (stocké dans mysql.tables_priv).*
nom_bdd.nom_routine	S'applique à la procédure (ou fonction) stockée *nom_bdd.nom_routine* (privilège stocké dans *mysql.procs_priv).*

Les privilèges peuvent aussi être restreints à certaines colonnes, auquel cas ils seront stockés dans la table *mysql.columns_priv*. Nous verrons comment restreindre un privilège à certaines colonnes avec la commande GRANT.

iii. AJOUT DE PRIVILEGES

Pour pouvoir ajouter un privilège à un utilisateur, il faut posséder le privilège GRANT OPTION.

Pour l'instant, seul l'utilisateur "root" le possède.

Connectez-vous donc avec "root" pour exécuter les commandes de cette partie.

SYNTAXE

```
GRANT privilege [(liste_colonnes)] [, privilege [(liste_colonnes)], ...]
ON [type_objet] niveau_privilege
TO utilisateur [IDENTIFIED BY mot_de_passe];
```

- privilege : privilège à accorder à l'utilisateur (SELECT, CREATE VIEW, EXECUTE…) ;
- (liste_colonnes) : facultatif - liste des colonnes auxquelles le privilège s'applique ;
- niveau_privilege : niveau auquel le privilège s'applique (*.*, *nom_bdd.nom_table*…) ;
- type_objet : en cas de noms ambigus, il est possible de préciser à quoi se rapporte le niveau, TABLE ou PROCEDURE.

On peut accorder plusieurs privilèges en une fois : il suffit de séparer les privilèges par une virgule. Si l'on veut restreindre tout ou partie des privilèges à certaines colonnes, la liste doit en être précisée pour chaque privilège.

Exemples

1. On crée un utilisateur 'cdpitbi'@'localhost', en lui donnant les privilèges SELECT, INSERT et DELETE sur la table parc_auto.Voiture, et UPDATE sur les colonnes immat, datecirc et pxachat de la table parc_auto.Voiture.

```
CREATE USER 'cdpitb1'@'localhost' IDENTIFIED BY 'colparsup2023';
GRANT SELECT,
      UPDATE (immat, datecirc, pxachat),
      DELETE,
      INSERT
ON parc_auto.Voiture TO 'cdpitb1'@'localhost';
```

2. On accorde le privilège SELECT à l'utilisateur 'cdpitb1'@'localhost' sur la table parc_auto.Voiture.

```
GRANT SELECT
ON TABLE parc_auto.Voiture -- On précise que c'est une table (facultatif)
TO 'cdpitb1'@'localhost';
```

iv. ➢ REVOCATION DE PRIVILEGES

Pour retirer un ou plusieurs privilèges à un utilisateur, on utilise la commande REVOKE.

```
REVOKE privilege [, privilege, ...]
ON niveau_privilege
FROM utilisateur ;
```

Exemple

```
REVOKE DELETE
ON parc_auto.Voiture
FROM 'cdpitb1'@'localhost';
```

d. PRIVILEGES PARTICULIERS

i. ALL

Le privilège ALL (ou ALL PRIVILEGES), comme son nom l'indique, représente tous les privilèges. Accorder le privilège ALL revient donc à accorder tous les droits à l'utilisateur. Il faut évidemment préciser le niveau auquel tous les droits sont accordés (on octroie tous les privilèges possibles sur une table, ou sur une base de données, etc.).

Un privilège fait exception : GRANT OPTION n'est pas compris dans les privilèges représentés par ALL.

Exemple : on accorde tous les droits sur la table *Client* à 'cdpitb1'@'localhost'.

```
GRANT ALL
ON parc_auto.Voiture
TO 'cdpitb1'@'localhost';
```

ii. USAGE

À l'inverse de ALL, le privilège USAGE signifie "aucun privilège". À première vue, utiliser la commande GRANT pour n'accorder aucun privilège peut sembler un peu ridicule. En réalité, c'est extrêmement utile : USAGE permet en fait de modifier les caractéristiques d'un compte avec la commande GRANT, sans modifier les privilèges du compte. Cet usage de la commande GRANT est cependant déconseillé.

USAGE est toujours utilisé comme un privilège global (donc ON *.*).

iii. GRANT OPTION

Nous voici donc au fameux privilège GRANT OPTION. Un utilisateur ayant ce privilège est autorisé à utiliser la commande GRANT, pour accorder des privilèges à d'autres utilisateurs.

Ce privilège n'est pas compris dans le privilège ALL.

Par ailleurs, un utilisateur ne peut accorder que les privilèges qu'il possède lui-même.

On peut accorder GRANT OPTION de deux manières :

- comme un privilège normal, après le mot GRANT ;
- à la fin de la commande GRANT, avec la clause WITH GRANT OPTION.

Exemple : on accorde les privilèges SELECT, UPDATE, INSERT, DELETE et GRANT OPTION sur la base de données *parc_auto* à 'admincdp'@'localhost'.

```
GRANT SELECT, UPDATE, INSERT, DELETE, GRANT OPTION
ON parc_auto.*
TO 'admincdp'@'localhost' IDENTIFIED BY 'ministrat12';

-- OU

GRANT SELECT, UPDATE, INSERT, DELETE
ON parc_auto.*
TO 'admincdp'@'localhost' IDENTIFIED BY 'ministrat12'
WITH GRANT OPTION;
```

Le privilège ALL doit s'utiliser tout seul. Il n'est donc pas possible d'accorder GRANT OPTION et ALL de la manière suivante : GRANT ALL, GRANT OPTION… Il est nécessaire dans ce cas d'utiliser WITH GRANT OPTION.